



**University of
Nottingham**

UK | CHINA | MALAYSIA

Sherlock AI: An Ethical Pipeline for LLM Character Emulation and role-play with an Interactive Web Platform

Submitted 06 September 2024, in partial fulfillment of
the conditions for the award of the degree **M.Sc Computer Science (Artificial
Intelligence)**.

Mustafa Mehmood
20306551

Supervised by Dr. Jeremie Cl

School of Computer Science
University of Nottingham

I hereby declare that this dissertation is all my own work, except as indicated in the
text:

Signature _____

Date ____ / ____ / ____

I hereby declare that I have all necessary rights and consents to publicly distribute this
dissertation via the University of Nottingham's e-dissertation archive.

Abstract

Recent advancements in Large Language Models (LLMs) have opened new possibilities for character emulation, yet challenges persist in creating authentic, context-aware AI personalities that maintain their persona over extended conversations. This project explores the development of an ethical pipeline for LLM character emulation, focusing on Sherlock Holmes as a case study. I fine-tuned a 4-bit quantized LLaMA 3.1 8B model using Parameter-Efficient Fine-Tuning (PEFT) techniques, specifically Low-Rank Adaptation (LoRA), to create an authentic Sherlock Holmes based conversational model. A user-friendly web application was developed using Flask and JavaScript to facilitate interaction with the AI, incorporating user-friendly features such as custom character creation, text-to-speech functionality, chat retrieval system and user management system. The model's performance was evaluated using both quantitative metrics (BLEU, cosine similarity, LIWC analysis) and qualitative user studies. A/B testing with 7 participants revealed a significant preference for the fine-tuned model (71.43%) over a baseline model, with higher ratings for authenticity (4.14 vs 3.29 on a 5-point scale) and logical deduction ability (4.43 vs 4.14). The web application also received high usability ratings (5/5 for navigation) in the usability testing. The project also addressed ethical considerations in AI character emulation by demonstrating a methodology for responsibly developing AI personalities using public domain characters.

Repository : <https://github.com/mustafameh/Sherlock-LLM>
GGUF Finetuned model download: https://drive.google.com/drive/folders/1SrBH66qNq7k65IUJy6nAFWot1w67B_lw?usp=sharing.
Webapp short demo video <https://youtu.be/K7Yxn6PkKmc>.

Acknowledgements

My deepest gratitude goes to Dr. Jeremie Clos, whose supervision and expertise were instrumental in shaping my project. His organization of regular group meetings fostered an environment of collaborative learning that proved invaluable. To my fellow group members, your insightful discussions and constructive feedback have been a constant source of inspiration and refinement for my work. I am particularly indebted to the seven students who volunteered their time for the user testing phase. Your engagement with Sherlock AI and your thoughtful feedback were key in the success of my project. I owe an immeasurable debt of gratitude to my parents. Their belief in me has been a constant source of motivation and encouragement.

Contents

Contents	5
1 Introduction	8
1.1 Background, Context, and Motivation	8
1.2 Problem Statement and Objectives	9
2 Literature Review	10
2.1 Introduction	10
2.2 Evolution of Language Models	11
2.3 Transfer=learning, Fine-tuning and Persona Consistency	12
2.4 Public Domain Characters: A Solution to Ethical Challenges in AI Training	14
3 Methodology	15
3.1 Overview	15
3.2 Model Development	17
3.2.1 Dataset Description and Structure	18
3.2.1.1 Dataset Composition:	18
3.2.1.2 Dataset Format	18
3.2.2 Data collection and Preprocessing.	21
3.2.2.1 Dialogue Extraction and Cleaning	22
3.2.2.2 Manual Curation, Quality Control and Annotation	23
3.2.3 Model Selection and Fine-tuning setup	23
3.2.3.1 Model Selection	23
3.2.3.2 Data Formatting	24
3.2.3.3 Parameter Fine-tuning Setup	24
3.2.4 Initial Model Performance Evaluation	27
3.2.4.1 Evaluation Metrics and Setup	27
3.2.4.2 Results and Initial Analysis	28
3.3 Software Development	29
3.3.1 User Requirements.	29
3.3.1.1 Functional Requirements.	29

3.3.1.2	Non-Functional Requirements	33
3.3.2	System Architecture Overview	34
3.3.3	Backend Implementation	35
3.3.3.1	Flask Server Setup and Configuration	36
3.3.3.2	API Design and Routing	37
3.3.3.3	Language Model Management and Inference Handling . .	38
3.3.3.4	Text-to-Speech Integration	39
3.3.3.5	User Authentication and Session Management	39
3.3.3.6	Custom Character backend	40
3.3.4	Database Design	41
3.3.5	Front-end Design and Implementation	44
3.3.5.1	Overall Design Philosophy and UX Goals	44
3.3.5.2	Layout of the Interface	44
3.3.5.3	User Authentication Pages	49
3.3.5.4	Front-end Implementation of Key Operations	51
4	Evaluation	67
4.1	Introduction to Evaluation	67
4.2	User Study	67
4.2.1	Study Design	68
4.2.1.1	Participants	68
4.2.1.2	Materials and Setup	68
4.2.1.3	Procedure	71
4.2.1.4	Measures and Metrics	73
4.2.1.5	Limitations	76
4.2.2	Results	76
4.2.2.1	A/B testing Results	76
4.2.2.2	Website Usability testing Results	84
4.3	Fine-tuned model Evaluation and Discussion	87
4.3.0.1	Quantitative Metrics vs. User Perceptions	87
4.3.0.2	Authenticity and Character traits	87
4.3.0.3	Context Adaptation and Persona	88
4.3.0.4	Creativity vs. Authenticity: A Delicate Balance	88

4.3.0.5	Challenges and limitations	88
4.3.0.6	Ethical Challenges and Broader Implication of Character Emulation	89
4.3.0.7	Future Directions	90
4.4	WebApp usability Evaluation and Discussion	90
4.4.0.1	Navigation and Core Functionality	90
4.4.0.2	Feature-Specific Usability	91
4.4.0.3	Challenges and Limitations	91
5	Conclusion and Reflection	92
	References	94
	Appendices	96

1 Introduction

1.1 Background, Context, and Motivation

In recent years, Large Language Models (LLMs) have revolutionized the landscape of artificial intelligence, pushing the boundaries of what's possible in AI-human interaction. As these models grow more sophisticated, a fascinating new frontier has emerged: the ability to emulate specific characters authentically, blending the realms of artificial intelligence, literature, and psychology. This project goes into this exciting intersection, exploring the potential of AI to breathe new life into not just beloved literary figures and historical personalities but but even ordinary people whose lives have been well-documented, allowing for a unique blend of reality and bringing their stories to life .

The allure of conversing with Sherlock Holmes , debating philosophy with Socrates, or exchanging witticisms with Oscar Wilde has long been confined to the realm of imagination. Recent advancements in LLMs have brought us tantalizingly close to making these interactions a reality. This possibility not only captivates the imagination but also opens up new avenues for education, entertainment, and cultural preservation. However, the true potential of this technology lies in its broader applications for human connection and memory preservation.

Perhaps most profoundly, the ability to clone conversational styles offers new possibilities for emotional support and legacy preservation. With proper ethical considerations and consent, this technology could help people cope with loss by interacting with a digital representation of a loved one, or allow future generations to "meet" ancestors they never knew. While the deployment of such technology must be handled carefully to avoid dependencies or prolonged grief, framing it as a memory preservation tool rather than a way to "bring someone back" could provide comfort and connection across time.

The potential applications extend beyond personal use. Businesses could leverage style-specific models to enhance customer satisfaction, boost sales, and improve service efficiency through tailored interactions. In education, students could engage with historical figures for immersive learning experiences. The entertainment industry could create new forms of interactive narratives.

The challenge lies in creating AI interactions that capture not just speech patterns, but the essence of an individual's unique personality, linguistic nuances, and thought processes. This involves integrating semantic and syntactic preferences effectively, aiming to embody the depth and complexity of human (or literary) personas.

While the project focuses on Sherlock Holmes due to ethical considerations and data availability (more on it in literature review), the ultimate aim is to create a pipeline that can be applied to emulate any personality or character, provided the data is ethically acquired. This should set a stage for responsible development in character emulation,

balancing innovation with ethical considerations. The choice of Sherlock Holmes as the subject is not arbitrary either – his complex personality, unique mode of thinking, and enduring popularity make him an ideal candidate for exploring the limits of character emulation.

1.2 Problem Statement and Objectives

Despite significant advancements in LLMs, my literature review has identified several critical gaps in creating an AI that can authentically emulate a specific character or individual. Current models often struggle to capture the unique linguistic nuances, reasoning patterns, and personality traits that define individual characters, creating a disconnect between generic AI outputs and truly personalized interactions. (see Section 2.3 for a detailed discussion on character emulation challenges)

Specifically, I’ve identified the following **key challenges**:

1. Lack of efficient methods for fine-tuning LLMs on limited computational resources, hindering accessibility for many researchers and developers.
2. Difficulty in maintaining persona consistency across extended conversations.
3. The challenge of balancing general language capabilities with character-specific traits.
4. Limited research on ethical frameworks for character emulation, particularly when using public domain characters.
5. Absence of a comprehensive, transferable pipeline that addresses both technical and ethical aspects of character cloning.

The project aims to address these challenges through the following **objectives**:

1. Develop a fine-tuned LLM capable of authentically emulating Sherlock Holmes over extended conversations, capturing his unique personality, deductive reasoning, and linguistic style.
2. Create an efficient and transferable fine-tuning pipeline operable on limited computational resources, making character emulation more accessible (building on PEFT techniques like LoRA, detailed in Section 2.3).
3. Design and implement a user-friendly web interface for interacting with the Sherlock Holmes. This platform will be intuitive and engaging, allowing users to explore the model’s capabilities. It will support both locally fine-tuned models and external APIs, offering flexibility in model usage. The interface will include features such as user management (e.g., sign-up and sign-in options), chat history for persistent

interactions, and integrated text-to-speech functionality to enhance the realism of the interactive experience.

4. Evaluate the model’s performance through multiple approaches: Utilize traditional NLP quantitative metrics like BLEU, cosine similarity , LIWC and sentiment analysis on both training and unseen data to assess the model’s learning and generalization capabilities. Conduct a comprehensive user study to evaluate how well the Sherlock Holmes model replicates the character’s traits and how users interact with the developed web application.
5. Ethically acquiring and using data for character emulation to set a precedent for responsible development guided by insights from the in-depth literature review on ethical considerations in data acquisition and usage, ensuring strict adherence to ethical guidelines.(see Section 2.4 for further discussion)

By choosing Sherlock Holmes, a public domain character, I navigate the ethical and legal challenges associated with using copyrighted or personal data. This approach allows me to focus on technical challenges while demonstrating a methodology that can be applied to a wide range of characters or personalities.

2 Literature Review

2.1 Introduction

This literature review aims to explore the current state of character emulation and authenticity in Large Language Models (LLMs), with a focus on fine-tuning techniques for conversational agents. The purpose is to establish a foundation for developing an accessible, ethical, and transferable pipeline for character cloning. The review covers several key areas, beginning with the evolution of Large Language Models, tracing their development from early transformer architectures to more recent, efficient models like LLaMA 3.1. It then examines fine-tuning techniques for conversational agents, including LORA and QLORA, which enable the adaptation of pre-trained models for specific tasks or characters. The review also goes into character emulation in conversational AI, examining principles, and techniques for maintaining authenticity. Ethical considerations and accessibility in character cloning are addressed, exploring the challenges and best practices in creating responsible AI characters. Finally, the review investigates the development of transferable pipelines for character cloning, exploring current approaches and challenges in creating generalizable methods. By critically analyzing these areas, this review aims to identify gaps in current research and highlight the need for further investigation into efficient, ethical, and accessible methods for character emulation using LLMs.

2.2 Evolution of Language Models

Language Models (LMs) are foundational to natural language processing, designed to predict the probability of sequences of words. Large Language Models (LLMs) represent a significant leap in this field, characterized by their massive scale and impressive capabilities across a wide range of language tasks. These models, at their core, are neural networks trained on vast corpora of text data, learning to understand and generate human-like text with remarkable accuracy and fluency.

LLMs began with the advent of deep learning techniques in NLP. Early milestones include the introduction of word embedding like Word2Vec Mikolov et al. (2013) and GloVe Pennington et al. (2014), which provided dense vector representations of words, capturing semantic relationships. However, the true revolution in LLMs came with the introduction of the Transformer architecture in the seminal paper "Attention Is All You Need" Vaswani et al. (2017). This work introduced the self-attention mechanism, allowing models to weigh the importance of different words in a sequence dynamically, leading to significant improvements in handling long-range dependencies in text.

Building on the Transformer architecture, subsequent models like BERT (Bidirectional Encoder Representations from Transformers) Devlin et al. (2018) demonstrated the power of pre-training on large corpora followed by fine-tuning on specific tasks. BERT's bidirectional nature allowed it to consider context from both directions, setting new benchmarks across various NLP tasks. The GPT (Generative Pre-trained Transformer) series, particularly GPT-3 Brown et al. (2020), further pushed the boundaries by scaling up model size to 175 billion parameters, showcasing impressive few-shot learning capabilities across a diverse range of tasks without task-specific fine-tuning.

However, the trend of increasing model size to achieve better performance came with significant challenges. The computational resources required to train and deploy these massive models were prohibitive for many researchers and organizations. This led to a shift in focus towards developing more efficient models that could maintain high performance with fewer parameters.

A pivotal moment in this evolution was the development of distillation techniques, exemplified by DistilBERT Sanh et al. (2019). This approach involved training a smaller model (the student) to mimic the behavior of a larger model (the teacher), effectively compressing the knowledge into a more compact form. DistilBERT managed to retain 95% of BERT's performance while reducing the parameter count by 40%, marking a significant step towards more efficient LLMs.

The quest for efficiency continued with the emergence of models like GPT-Neo Black et al. (2021) and GPT-J Wang et al. (2021). These open-source alternatives to proprietary models like GPT-3 demonstrated that careful architectural and training optimizations could yield highly effective smaller models. This development was important in democratizing

access to powerful LLMs, catering to the growing demand for accessible and manageable AI tools. (lacking proper citations from academic sources)

A significant leap forward in this direction came with Meta’s LLaMA series. The LLaMA models Touvron et al. (2023), particularly LLaMA 2 and the recent LLaMA 3.1AI (2024), exemplify how innovative training techniques and architectural adjustments can produce smaller models that compete with much larger ones. LLaMA 3.1, with its 8B parameter variant, employs techniques like Grouped-Query Attention (GQA) to achieve performance levels previously associated with models many times its size. Its ability to match or surpass larger models like GPT-3.5 in tasks such as general problem-solving and multilingual capabilities represents a paradigm shift in LLM development.

Similarly, models like GEMMA 2.1 Riviere et al. (2024) focus on streamlining the Transformer architecture to enhance performance and reduce latency in real-time applications. These developments, along with smaller variants of models like Mistral Jiang et al. (2023), demonstrate that reduced-scale models can effectively handle complex language tasks, challenging the notion that bigger is always better in the world of LLMs.

The shift towards smaller, more efficient models is not just about reducing computational costs; it’s about democratizing access to AI technologies. As models become more efficient and accessible, the potential for research on creating authentic, character-specific conversational agents increases. This trend aligns closely with the goals of creating accessible and transferable pipelines for character cloning, allowing for more widespread experimentation and application of these technologies in various fields and by a diverse range of users.

However, challenges remain in adapting these models to capture the nuances of individual characters while maintaining general language understanding capabilities. As we move forward, the focus on LLM development is likely to continue balancing performance with efficiency, accessibility, and adaptability. This shift sets the stage for a more sophisticated approach to character emulation., where the goal is not just to generate coherent text but to capture the essence of specific personalities in a computationally efficient manner. To tackle these hurdles and unlock the full potential of LLMs in bringing characters to life, researchers are exploring advanced fine-tuning techniques, which will be examined in the next section.

2.3 Transfer=learning, Fine-tuning and Persona Consistency

Fine-tuning is an important phase in deploying large Language Models (LLMs), adapting pre-trained models to specific tasks or domains to enhance performance. This process utilizes transfer learning, where knowledge from one problem is applied to related problems, enabling LLMs to overcome generic training limitations and perform nuanced understanding and generation. As LLMs have evolved, efficient fine-tuning has become

essential, especially for specialized applications like character emulation. It allows models to adapt without retraining from scratch, conserving time and computational resources. Transfer learning in LLMs involves two key stages: pre-training on diverse texts to learn general language patterns, and fine-tuning, which tailors this knowledge to specific tasks. Raffel et al. (2020) highlighted this in "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," demonstrating that a pre-trained model could excel across multiple NLP tasks by treating them as text-to-text problems. They introduced the T5 (Text-to-Text Transfer Transformer) model, demonstrating that a single pre-trained model could be fine-tuned to perform well on a wide variety of NLP tasks. The authors showed that by framing all NLP problems as text-to-text tasks, they could achieve state-of-the-art results on many benchmarks while providing a unified framework for transfer learning in NLP.

Houlsby et al. (2019) introduced a new wave of efficiency in "Parameter-Efficient Transfer Learning for NLP," embedding small trainable modules within a model's layers for task-specific fine-tuning. These adapters, updating just 3.6% of BERT's parameters, achieved full fine-tuning performance and setting the stage for subsequent advancements like LoRA. As models expanded, traditional fine-tuning became costly and prone to catastrophic forgetting. Addressing this, Hu and other authors (2021) in "LoRA: Low-Rank Adaptation of Large Language Models," developed LoRA, which integrates trainable rank decomposition matrices into the Transformer architecture, drastically cutting trainable parameters and maintaining performance while updating just 0.01% of parameters.

Further enhancing LoRA's efficiency, Dettmers et al. (2023) introduced QLoRA in "QLoRA: Efficient Finetuning of Quantized LLMs," merging quantization with LoRA to allow fine-tuning on a single GPU. By implementing 4-bit quantization and "paged optimizers," QLoRA reduces memory demands while maintaining model quality. This advancement is particularly relevant for character emulation tasks, as it allows researchers with limited computational resources to fine-tune large models for specific characters or conversational styles.

Despite these advancements, fine-tuning LLMs for character emulation presents unique challenges. Madotto et al. (2021) introduced "Few-Shot Persona-Grounded Dialogue via Contrastive Fine-Tuning" that specifically targets the challenges of generating persona-consistent dialogues in conversational AI with limited training data. Their approach utilizes contrastive learning—a technique that learns to distinguish between similar and dissimilar examples—to fine-tune language models. This method not only enhances the model's ability to generate responses that align with a given character's persona using fewer examples but also significantly improves the consistency of these persona traits across different dialogues.

Furthermore, Huang et al. (2022) addressed the difficulty of sustaining persona attributes over extended conversations. They introduced a method called Persona-Adaptive Atten-

tion (PAA), which dynamically adjusts the weighting of persona and dialogue context information during the conversation. This technique allows the model to maintain a strong persona while adapting to the flow of the dialogue, enhancing character consistency without compromising the naturalness of the interaction. It also highlighted the delicate balance required between general language proficiency and persona-specific nuances.

The study on generating persona-consistent dialogue responses using deep reinforcement learning further enriches this discussion by introducing a methodological framework that actively rewards persona-consistent responses, thereby ensuring that the AI's output remains aligned with the predefined character traits throughout the interaction Simpson (2020).

As we move forward, the integration of these efficient fine-tuning techniques with character-specific training strategies presents a promising avenue for creating more authentic and computationally accessible AI characters. However, challenges remain in achieving the right balance between general language capabilities and character-specific traits, as well as in ensuring the ethical use of such technologies.

2.4 Public Domain Characters: A Solution to Ethical Challenges in AI Training

While the technical advancements in LLMs, fine-tuning and other techniques have opened new possibilities for character emulation, they also raise significant ethical and legal concerns. The use of copyrighted material in AI training has become a contentious issue, as highlighted by Johnson and Yi (2024) in their analysis of copyright infringement risks in AI training. They emphasize the potential legal challenges associated with scraping data from the internet and stress the importance of careful data sourcing and adherence to fair use principles.

The specific case of generative AI systems like ChatGPT, as examined by the European Journal of Risk Regulation (2023), further underscores the complex interplay between copyright law and AI development. This study explores the ethical and legal dilemmas surrounding the use of large datasets, proposing strategies to balance the protection of stakeholder rights with the advancement of AI technology. The American Bar Association (2024) echoes these concerns, emphasizing the need for a clear legal framework that fosters innovation while respecting copyright holders' interests.

In light of these challenges, the project adopts a pragmatic approach by utilizing open-source, public domain characters like Sherlock Holmes for fine-tuning LLM. This decision serves multiple purposes: it avoids potential copyright infringement, demonstrates versatility of the pipeline in handling different character types, and provides, well-documented character for emulation. This allows focus on tackling key challenges in the field, such

as maintaining persona consistency and capturing nuanced character traits. By using a character in the public domain, a replicable and ethically sound methodology is created that other researchers can adopt or adapt for their own studies, including those involving copyrighted or personal data, should they have the necessary permissions.

This approach aligns with the goal of creating an accessible and transferable pipeline for character cloning while addressing multiple gaps in current research. By focusing on public domain characters like Sherlock Holmes, I try to navigate ethical and legal challenges and also create a robust testbed for tackling key technical issues in character emulation. This strategy allows us to concentrate on improving fine-tuning techniques for enhanced persona consistency, as highlighted by Madotto et al. (2021), and developing methods for maintaining character traits over extended conversations, building on the work of Huang et al. (2022). Moreover, using a well-documented character enables us to rigorously evaluate our model’s ability to capture nuanced personality traits and maintain authenticity in diverse conversational contexts. The work sets a foundation for future work that can be applied to a wide range of characters, both fictional and real, advancing character emulation while maintaining ethical integrity.

3 Methodology

3.1 Overview

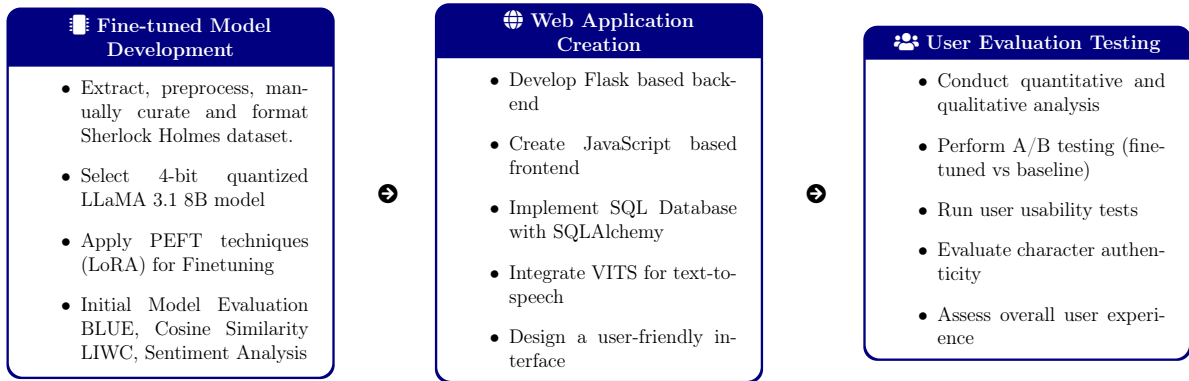


Figure 1: Methodology Overview Diagram

The methodology encompasses three main components: fine-tuned model development, web application creation, and user evaluation testing. It begins with the creation of a meticulously curated dataset, comprising conversational exchanges featuring Sherlock Holmes primarily derived from Sir Arthur Conan Doyle’s original stories to maintain authenticity. This data underwent rigorous preprocessing, including cleaning, segmentation, and manual annotation, to capture Holmes’ unique characteristics and conversational

style. The dataset was then formatted into a structured prompt format suitable for model training.

For the model, a 4-bit quantized version of the LLaMA 3.1 8B model was selected as the base, chosen for its efficiency and open-source nature. The fine-tuning process utilized Parameter-Efficient Fine-Tuning (PEFT) techniques, specifically Low-Rank Adaptation (LoRA), implemented using the Hugging Face Transformers library and the Unsloth framework for optimization. This approach ensured the model remained lightweight and accessible, suitable for training and deployment in environments with limited computational resources. Upon completion of training, the BLEU scores, cosine similarity score, sentimental analysis and LIWC metrics were calculated from model inferences, applied to both the training data and a set of unseen data to verify the model’s learning and generalization capabilities and get an initial evaluation of its capabilities..

The web application was developed using Flask for the backend and vanilla JavaScript for the frontend, supporting both local inference and external API integration for model flexibility. Flask manages server-side operations including routing, model management, user authentication, database interactions, and text-to-speech conversion. The JavaScript frontend handles user interface elements, API communication, chat flow, and audio playback. An SQL database, managed through SQLAlchemy, stores user data and chat history for persistent interactions. The VITS library was integrated for text-to-speech functionality, enhancing the interactive experience. Tailwind CSS was employed for a responsive and modern interface design.

Evaluation of the model’s performance employed both quantitative metrics and qualitative analysis. I conducted A/B testing, comparing the fine-tuned model against a baseline non-fine-tuned LLaMA 3.1 8B model, assessing both character authenticity and user experience. Following the A/B testing, the original web application underwent user usability tests to further refine and validate the user interface and overall experience.

The entire methodology was designed to create an accessible, efficient, and authentic character emulation system that can be applied to a wide range of characters, both fictional and real.

3.2 Model Development

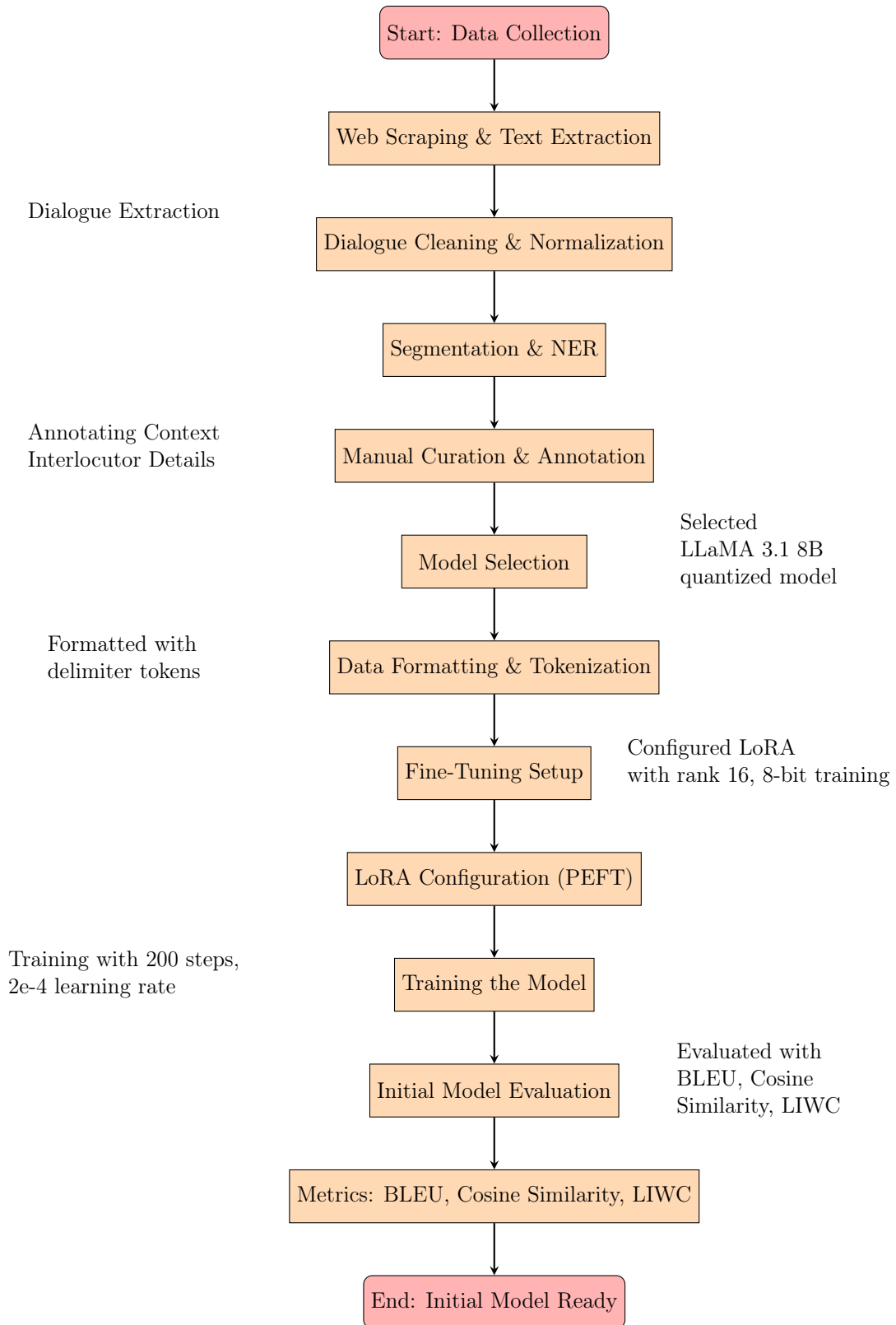


Figure 2: Enhanced Model Development Workflow

3.2.1 Dataset Description and Structure

The dataset comprises 230 unique conversational exchanges which was meticulously curated to authentically capture the character as portrayed in Sir Arthur Conan Doyle’s original stories. . The aim was to capture the essence of Holmes’ character, including his character traits, deductive reasoning, speech patterns, and interactions with various characters from the Sherlock Holmes universe.

3.2.1.1 Dataset Composition:

The dataset was primarily based on dialogues extracted from the original Sherlock Holmes stories, with the majority being direct quotes to ensure a high level of authenticity. For better alignment with conversational contexts, some dialogues were slightly paraphrased, and a few were newly created in Holmes’ style to cover scenarios not present in the original texts. See Fig.3

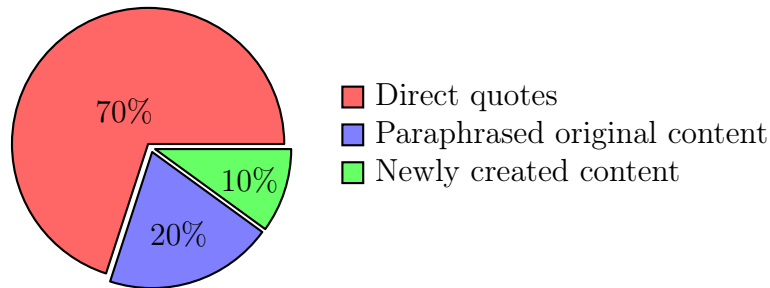


Figure 3: Dataset Composition

3.2.1.2 Dataset Format

The dataset was structured as a JSON array, (See Listing 1) with each entry representing a distinct conversation (See Listing 2) . Each conversation consisted of varying numbers of back-and-forth exchanges to train the model in handling both brief and extended conversations. See Fig.4.

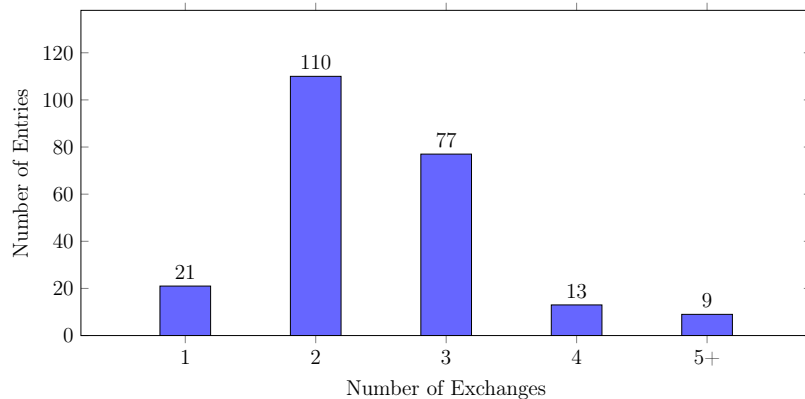


Figure 4: Number of Entries by Number of Exchanges (Message Pairs)

The conversations were formatted to include the following components

1. System Message:

- Appears once at the beginning of each conversation and provides instructions for responding in character as Sherlock Holmes
- Typically includes:
 - A concise description of Holmes’ character, encompassing his keen observational skills, logical reasoning, direct and analytical speaking style, confidence, occasional dramatic flair, and tendency for sharp or sarcastic remarks, as well as his often aloof or detached emotional disposition
 - Context for the specific conversation, such as the setting or case background
 - Description of the interlocutor (the person Holmes is speaking to), including their relationship to Holmes and any relevant characteristics of that person.
- By including character traits, conversation context, and interlocutor details, it enables the model to generate responses that are not only consistent with Holmes’ character but also appropriate to the specific scenario and tailored to the particular person he’s interacting with. This should allow the model to learn to adjust Holmes’ conversational style and behavior dynamically, mirroring how a real person might interact differently with, for example, a close friend like Dr. Watson versus a client or a suspect in a case.

2. User and Assistant Messages:

- Follows the system message in alternating pairs where ‘user’ messages represent the dialogue of other characters interacting with Holmes. This can be questions, statements, or descriptions of actions and vary in length and complexity to mimic real conversations. ‘assistant’ messages represent Holmes’ responses showcasing Holmes’ deductive reasoning, observational skills, and characteristic speech patterns
- The number of message pairs can vary (typically 1 to 5) to represent different conversation lengths

This structure was deliberately followed to ensure compatibility and alignment with the format expected by the LLaMA 3.1 model for smooth training and inference processes

Listing 1: JSON Template for Dataset

```
[{
```

```
"messages": [  
  {  
    "role": "system",  
    "content": "[Character description] + [Context] + [Interlocutor  
      description]"  
  },  
  {  
    "role": "user",  
    "content": "Interlocutor's message"  
  },  
  {  
    "role": "assistant",  
    "content": "Sherlock Holmes' response"  
  }  
  // Additional message pairs...  
]  
},  
// Next conversation]
```

Listing 2: Example Conversation Entry

```
{  
  "messages": [  
    {  
      "role": "system",  
      "content": "You are Sherlock Holmes, the famous detective known  
        for your proficiency in observation, deduction, forensic  
        science, and logical reasoning. Respond in character. Holmes  
        typically speaks in a direct, analytical manner, often  
        impatient with those who can't follow his rapid thought  
        processes. Context: You are in your study at 221B Baker  
        Street, sharing your insights on a recent case about a missing  
        gem. You are currently in a conversation with Dr. John Watson,  
        Holmes's loyal friend and biographer, who speaks with earnest  
        curiosity and a deep respect for Holmes's abilities, often  
        providing a more human and emotional perspective to Holmes's  
        detached and analytical nature."  
    },  
    {  
      "role": "user",  
      "content": "But how did you deduce that the gem was still in the  
        goose, Holmes?"  
    },  
    {  
      "role": "assistant",  
      "content": "By the simple process of elimination, my dear Watson.  
        The thief would not have left such a valuable prize with the  
        seller, nor would the seller have knowingly parted with it."  
    }  
  ]  
}
```

```

    Therefore, it must have remained with the bird unbeknownst to
    all until we discovered it."
  },
  {
    "role": "user",
    "content": "And yet, you didn't immediately pursue the man who
    lost the gem?"
  },
  {
    "role": "assistant",
    "content": "Patience, Watson. The gem is safe in our hands now,
    and the true culprit will reveal himself in time, as
    desperation often leads to foolish actions."
  }
}

```

Test Set

A subset of approximately 15% of the dataset (30 entries) was set aside as a dedicated test set. It was used for calculating quantitative metrics such as BLEU and cosine similarity scores during one of the model evaluation phases to measure the model's output against reference responses, in order to assess model generalization.

3.2.2 Data collection and Preprocessing.

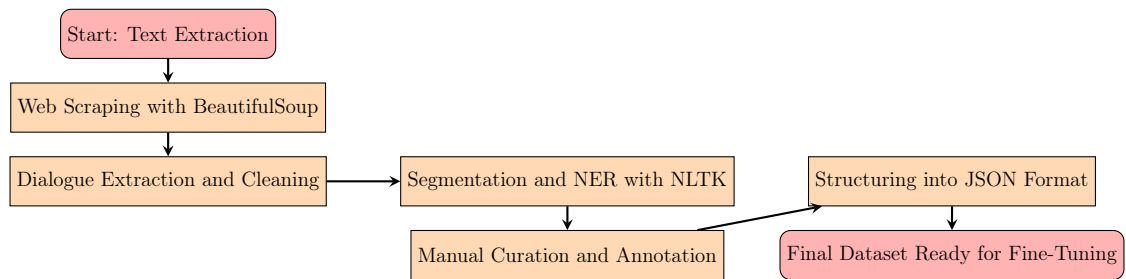


Figure 5: Flowchart of Dataset Preparation Process

Custom Python scripts were developed to facilitate the process of collecting and preprocessing data. It involved several steps, combining automated techniques with meticulous manual curation and annotation ensuring a high-quality dataset is created that accurately captures the essence of Holmes' character.

Text Extraction: The primary source of data was Sir Arthur Conan Doyle's original Sherlock Holmes stories, accessed through Project Gutenberg Doyle (2003). I employed web scraping techniques using the BeautifulSoup library Richardson (2007) to extract relevant text from these digital versions. The focus was on dialogues and scenarios that best showcase Holmes' unique character traits, deductive reasoning processes, and interactions with various characters.

Listing 3: Example Web Scraping with BeautifulSoup

```
1 from bs4 import BeautifulSoup
2 import requests
3 url = "https://www.gutenberg.org/files/1661/1661-h/1661-h.htm"
4 response = requests.get(url)
5 soup = BeautifulSoup(response.content, 'html.parser')
6 text = soup.get_text()
```

3.2.2.1 Dialogue Extraction and Cleaning

The extracted text underwent rigorous cleaning, segmentation, and named entity recognition to ensure data quality and enrich with metadata: a) Dialogue Extraction and Cleaning: I used regular expressions (regex) in Python to remove non-standard characters, excessive whitespace, and formatting issues. The clean-text library Scaffidi (2021) was utilized for additional text normalization. This ensured that only the most relevant parts of the text are captured while removing any inconsistencies or irrelevant information. b) Segmentation and Named Entity Recognition: Natural Language Toolkit (NLTK) Bird et al. (2009) was also employed for sentence segmentation and Named Entity Recognition (NER). This allowed me to divide the text into meaningful units and identify important entities within the dialogues, which later helped with metadata annotation in the system messages.

Listing 4: Example Dialogues Extraction Cleaning and NER

```
1 import re
2 from cleantext import clean
3 from nltk import sent_tokenize, ne_chunk, pos_tag, word_tokenize,
4 dialogues = re.findall(r' "(.*?)" ', text)
5 cleaned_dialogues = []
6 for dialogue in dialogues:
7     cleaned_dialogue = clean(
8         dialogue, fix_unicode=True,
9         to_ascii=True, lower=False,
10        no_line_breaks=True, no_numbers=False,
11        no_digits=False, no_punct=False,
12        lang="en"
13    )
14    cleaned_dialogues.append(cleaned_dialogue.strip())
15 def process_dialogue(dialogue):
16     sentences = sent_tokenize(dialogue)
17     entities = []
18     for sentence in sentences:
19         words = word_tokenize(sentence)
20         tagged = pos_tag(words)
21         named_entities = ne_chunk(tagged)
```

```
22     entities.extend([ne for ne in named_entities if isinstance(  
        ne, nltk.Tree)])  
23     return sentences, entities
```

3.2.2.2 Manual Curation, Quality Control and Annotation

A significant portion of the work involved manual curation and annotation to ensure relevance and highest quality of data:

- **Manual Review:** Each extracted piece was carefully reviewed for relevance and quality. Special attention was given to selecting dialogues and scenarios that best showcased Holmes’ character traits, deductive reasoning, and interactions with various characters. Care was taken to avoid offensive or inappropriate material while maintaining the historical context of the stories .
- **Manual Annotation:** Metadata such as story context, setting, and character information was added to system message of each conversation.

The processed and annotated data was structured into a JSON format as detailed above using the pandas library and an automated check was implemented to verify the structural integrity of the JSON format.

3.2.3 Model Selection and Fine-tuning setup

3.2.3.1 Model Selection

The foundation of the fine-tuning process was the **LLaMA 3.1** model with 8 billion parameters . To align with the project’s goal of accessibility, I opted for a **4-bit quantized** version of this model in order to reduce reduces the memory footprint and computational requirements even further. Quantization works by reducing the precision of model weights from 32-bit floating points to lower bit-width representations (like 4-bit integers), which significantly reduces the storage size—as in an 8 billion parameter model, this change decreases the size from around 30 GB to just 3.73 GB by using only 4 bits (or 0.5 bytes) per parameter instead of 32 bits (or 4 bytes). However, this typically incurs a small performance drop, often around 1-2% in accuracy depending on the task and model architecture Gholami et al. (2021). As part of the fine-tuning process, the text must first undergo tokenization, which is the conversion of text into numerical tokens that the model can process effectively.

I initialized both the model and tokenizer for the quantized LLaMA 3.1 8B model using FastLanguageModel from *unsloth* Contributors (2023) , a Python library which provides great memory optimizations for training and finetuning.:

Listing 5: Model and Tokenizer Initialization

```
1 model, tokenizer = FastLanguageModel.from_pretrained(  
2 model_name = "unsloth/Meta-Llama-3.1-8B-Instruct-bnb-4bit",  
3 max_seq_length = max_seq_length,  
4 dtype = None, # Auto-detect  
5 load_in_4bit = True,  
6 )
```

3.2.3.2 Data Formatting

To align with LLaMA’s expected input format, I structured the training data using specific delimiter tokens. The delimiter tokens are important for the model to distinguish between different parts of the conversation, such as system messages, user inputs, and Sherlock’s responses. These tokens help the model understand the structure of the input, which is essential for generating coherent and contextually appropriate responses. The `<|eot_id|>` token, in particular, signals the end of each text segment, helping the model understand where one part of the conversation ends and another begins. This is also how it learns when to step generating text.

Listing 6: Data Preparation Function

```
1 def prepare_training_data(conversations):  
2     formatted_data = []  
3     for conversation in conversations:  
4         full_prompt = "<|begin_of_text|>"  
5         for message in conversation['messages']:  
6             role = message['role']  
7             content = message['content']  
8             full_prompt += f"<|start_header_id|>{role}<|  
9                 end_header_id|>\n{content}<|eot_id|>\n"  
10            formatted_data.append({"text": full_prompt})  
11    return formatted_data
```

3.2.3.3 Parameter Fine-tuning Setup

Fine-tuning techniques allow us to adapt the pre-trained LLaMA 3.1 model to specific tasks, such as emulating Sherlock Holmes, by adjusting its pre-trained weights based on a curated dataset. To fine tune the model I used Parameter-Efficient Fine-Tuning (PEFT), specifically leveraging Low-Rank Adaptation (LoRA) technique.

Why LoRA? LoRA is a method that adds trainable, low-rank matrices to certain layers of the model, significantly reducing the number of parameters that need to be updated during fine-tuning, making it ideal for scenarios with limited computational resources (See Literature Review for a detailed discussion on LoRA and other PEFT techniques).

The number of parameters updated by LoRA can be calculated as:

$$\text{Reduction Ratio} = \frac{\text{Updated Parameters}}{\text{Original Parameters}} = \frac{2 \times r \times d \times L}{N}$$

where:

- r : Rank of the low-rank adaptation.
- d : Dimension of the model's hidden size
- L : Number of layers in the model .
- N : Total number of parameters in the original model (e.g., 8 billion for my case).

This equation demonstrates how LoRA reduces the number of parameters that need to be updated, as the fraction $\frac{2 \times r \times d \times L}{N}$ represents a significantly smaller subset of the original model's parameters. Rank refers to the dimensionality of the low-rank matrices added to the model's layers, determining the degree to which these matrices can approximate changes in the original model's weights during fine-tuning.

Here's how I configured LoRA for my model:

Listing 7: LoRA Configuration

```
1 model = FastLanguageModel.get_peft_model(  
2 model,  
3 r = 16,  
4 target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",  
5 "gate_proj", "up_proj", "down_proj",],  
6 lora_alpha = 16,  
7 lora_dropout = 0,  
8 bias = "none",  
9 use_gradient_checkpointing = "unsloth",  
10 random_state = 3407,  
11 use_rslora = False,  
12 )
```

Key LoRA parameters:

- **r = 16**: Rank of the LoRA update matrices
- **target_modules**: Specifies which layers to apply LoRA to, including attention and MLP components
- **use_gradient_checkpointing**: A memory-saving technique that allows back-propagation through intermediate activations without storing them all, which is for handling model with limited resources.

Then I used the SFTTrainer from the trl (Transformer Reinforcement Learning) library von Werra et al. (2020) for training process which was set up the following configuration

Listing 8: SFTT Configuration

```
1  trainer = SFTTrainer(  
2      model = model,  
3      tokenizer = tokenizer,  
4      train_dataset = dataset,  
5      dataset_text_field = "text",  
6      max_seq_length = max_seq_length,  
7      dataset_num_proc = 1,  
8      packing = False,  
9      args = TrainingArguments(  
10         per_device_train_batch_size = 1,  
11         gradient_accumulation_steps = 2,  
12         warmup_steps = 2,  
13         max_steps = 200,  
14         learning_rate = 2e-4,  
15         fp16 = not torch.cuda.is_bf16_supported(),  
16         bf16 = torch.cuda.is_bf16_supported(),  
17         logging_steps = 1,  
18         optim = "adamw_8bit",  
19         weight_decay = 0.01,  
20         lr_scheduler_type = "linear",  
21         seed = 3407,  
22         output_dir = "outputs",  
23     ),  
24 )
```

Key training parameters:

- Learning rate: 2e-4 with a linear scheduler
- Optimizer: AdamW 8-bit for memory efficiency
- Mixed precision training: Using bfloat16 where supported
- Total training steps: 200

The training completed 3.74 epochs of the dataset in 200 steps and took about 30 minutes. Training loss started around 2.9 and decreased to about 0.15 by the end of training, indicating significant learning. A merged 16-bit version of the model was saved locally which was then converted in 8bit. For the entire fine-tuning process I ensured that the fine-tuned pipeline remains relatively lightweight and can be deployed on consumer grade systems with limited resources, such as the RTX4060 8GB GPU, 16GB RAM, and i7 13th gen CPU configuration used in this project.

3.2.4 Initial Model Performance Evaluation

Overview

I conducted an initial quantitative assessment of the fine-tuned model's performance prior to user-based evaluations. This preliminary evaluation aimed to verify the model's learning and provide insights into its basic capabilities, serving as a starting point rather than the primary system evaluation.

The assessment focused on confirming the model's learning from training data and gauging its ability to generate Sherlock Holmes-like responses. I utilized both the training data and a held-out test set of 30 conversations. The process involved generating responses from the model using a temperature setting of 0.7 to balance creativity and coherence, then comparing these responses to reference responses using various metrics.

3.2.4.1 Evaluation Metrics and Setup

I employed the following metrics to assess the quality of generated responses:

- **BLEU Scores:** Measures the similarity between a generated response and a reference response by evaluating how closely the word choices and order in the generated response match those in the reference response. (Implemented using NLTK's `sentence_bleu` function)
- **Cosine Similarity:** Calculates semantic similarity (meaning) between generated responses and reference text regardless of their length. (Utilized sklearn's `TfidfVectorizer` and `cosine_similarity` function)
- **Custom LIWC Analysis:** Evaluates the presence of specific linguistic categories relevant to Sherlock Holmes' character. I implemented a simplified version of the Linguistic Inquiry and Word Count (LIWC) approach, using WordNet to generate expanded word lists for key categories like cognitive processes(to capture analytical and deductive reasoning), analytical thinking (to reflect his logical approach to problem-solving), perception(to represent observational skills), emotion (to gauge the level of emotional expression) and social process (to assess his interaction and communication style).
- **Sentiment Analysis:** Assesses the emotional tone of generated responses. (Used NLTK's `SentimentIntensityAnalyzer`)

3.2.4.2 Results and Initial Analysis

Table 1: Summary of Model Performance on Training and Unseen Test Sets

Metric	Training Set	Unseen Test Set
Avg. BLEU Score	0.52 ± 0.34	0.126 ± 0.022
% High BLEU (0.75)	36.84%	6.67%
Avg. Cosine Similarity	0.65 ± 0.26	0.32 ± 0.237
% High Cosine (0.85)	34.91%	13.33%
Avg. Sentiment Score	0.20 ± 0.56	0.24 ± 0.36

Table 2: Sentiment Analysis Results

Metric	Training Set	Unseen Data
Range	-0.98 to 0.95	-0.25 to 0.86
Negative Responses (%)	27.10	24.00
Neutral Responses (%)	10.28	8.67
Positive Responses (%)	62.62	67.33

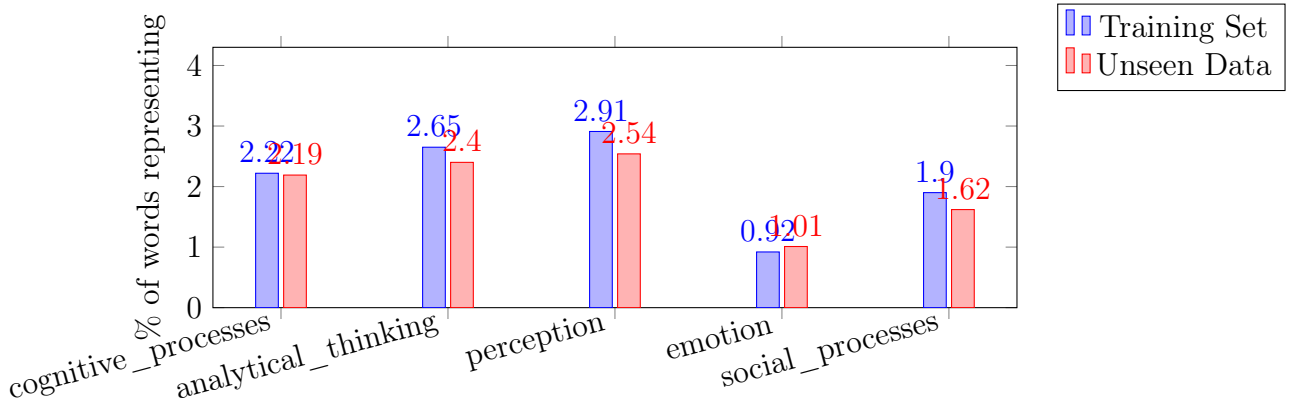


Figure 6: Comparison of LIWC Categories between Training Set and Unseen Data

On the training dataset, the moderate average BLEU score (0.52) as shown in Table 1 suggests that the model has learned to reproduce some of Holmes’ exact phrasings, while also introducing variations. This is desirable, as it indicates the model isn’t simply memorizing responses. The good average cosine similarity (0.65) indicates that the model is capturing the semantic essence of Holmes’ responses well, even when not using identical wording. The LIWC analysis score (as shown in Figure 6) results suggest the model is emphasizing analytical and perceptive language, aligning with Holmes’ character. The lower emotion score is consistent with Holmes’ typically detached demeanor. The slight positive skew with a wide range (-0.98 to 0.95) for sentiment analysis (shown in Table 2 suggests the model can generate responses with varied emotional tones, reflecting the diverse situations Holmes usually encounters.

For the unseen dataset, the notably lower BLEU (0.126) and cosine similarity (0.32) scores for unseen data, are primarily due to the model not knowing the specific stories or contexts of these new scenarios. Unlike with the training data, where the model has encountered similar situations before, the unseen data presents entirely new narratives. A creative adaptation is precisely what is aimed for in character emulation, as it allows the model to engage with users in new, unexpected scenarios while staying true to Holmes' essence. The LIWC and sentiment analyses show remarkable consistency with the training set, suggesting that the model maintains aspects of Holmes' character even in new situations. The model maintains key aspects of Holmes' linguistic style, emotional range, and thought patterns even when faced with entirely new scenarios.

However, this initial assessment also demonstrates the complexity of evaluating character emulation. While these metrics provide valuable insights, they don't fully capture the perceived realism or other nuances in character portrayal. This shows the importance of the subsequent user study and A/B testing, which will further enrich the evaluation of the model's effectiveness in true character emulation across diverse scenarios by taking into account human perception and interaction.

3.3 Software Development

The software development phase of this project focused on creating a user-friendly, interactive web application that brings the Sherlock Holmes AI model to life. This section details the process of designing and implementing a full-stack application that seamlessly integrates the fine-tuned AI model (and the flexibility of using other models) with a responsive user interface. The development prioritized accessibility, ensuring that users can engage with the Sherlock Holmes chatbot through an intuitive platform. The approach combined web technologies with AI integration techniques to create a robust, scalable solution. The following subsections will explore the user requirements that guided the development, the system architecture that forms the backbone of the application, and the specific implementations across backend, frontend, and database components.

3.3.1 User Requirements.

3.3.1.1 Functional Requirements.

Describe the specific behaviors and features of the Sherlock Holmes AI Chatbot application:

1. User Authentication and Management

- **User Registration:** Allow new users to create an account with a unique username and password.

- **User Login:** Enable registered users to securely log in to access additional feature of saving chats.
- **User Logout:** Provide a secure logout functionality for users to end their session.
- **Guest Access:** Allow users to use the application without logging in.

2. Conversational

- **Real-time Chat:** Enable users to engage in text-based conversations with an emulated version Sherlock Holmes .
- **Message History Display:** Show the ongoing conversation, including user inputs and AI responses.
- **Context Retention:** Maintain conversational context for coherent interactions.
- **Character Selection:** Allow users to choose from predefined characters (e.g., Dr. Watson, Mrs. Hudson) for role-play.
- **Text Messaging Style:** Design the chat interface to resemble a text messaging conversation with Sherlock Holmes.
- **Generation Indicator:** Display a visual indicator while the AI is generating a response.

3. AI Model Integration

- **Local Model Support:** Implement local inference using the fine-tuned Sherlock Holmes AI model.
- **External API Integration:** Provide option to use external AI model services (via OpenRouter) for model inference.
- **Model Switching:** Allow users to switch between local and external AI models.
- **Inference Settings:** Enable users to adjust AI parameters like temperature.
- **Custom API Key:** Allow users to input their own API key for online models.

4. Audio Features

- **Text-to-Speech Conversion:** Convert AI-generated text responses into audio.
- **Audio Playback:** Implement an integrated audio player for AI responses within the chat bubble to increase realism and interactivity.

- **Playback Controls:** Provide basic audio controls (play, pause, stop).

5. Custom Character Creation

- **Character Editor:** Implement an interface for users to create custom characters for role-play.
- **Character Attributes:** Allow users to define traits, speaking styles, and relationships for custom characters.
- **Custom Character Selection:** Make custom characters available for selection in conversations.

6. Data Management and Persistence

- **Manual Chat Saving:** Provide a button for users to save their current chat to their account.
- **Chat Retrieval:** Allow logged-in users to access and review their saved conversations.
- **Chat Deletion:** Enable users to delete their saved chats.
- **Private Access:** Ensure that saved chats are only accessible to the user who created them.

7. Settings and Customization

- **Settings Panel:** Implement a panel for users to customize their experience.
- **Options:** Provide options for switching between various models, setting the API key, logging-in and logging out, unloading/unloading local model, switching role play character, opening the custom character creation window, setting the temperature and finally toggling the API Window display.
- **Optional Context Window:** Include an optional context input field for users to provide additional background for the conversation.

8. API Window and Toggle

- **API Inspection:** Implement a toggle functionality to show/hide an API window.
- **API Window:** Display the exact message being sent to the API/model in this window.



Figure 7: Features Mindmap Diagram for the Sherlock Holmes AI Chatbot

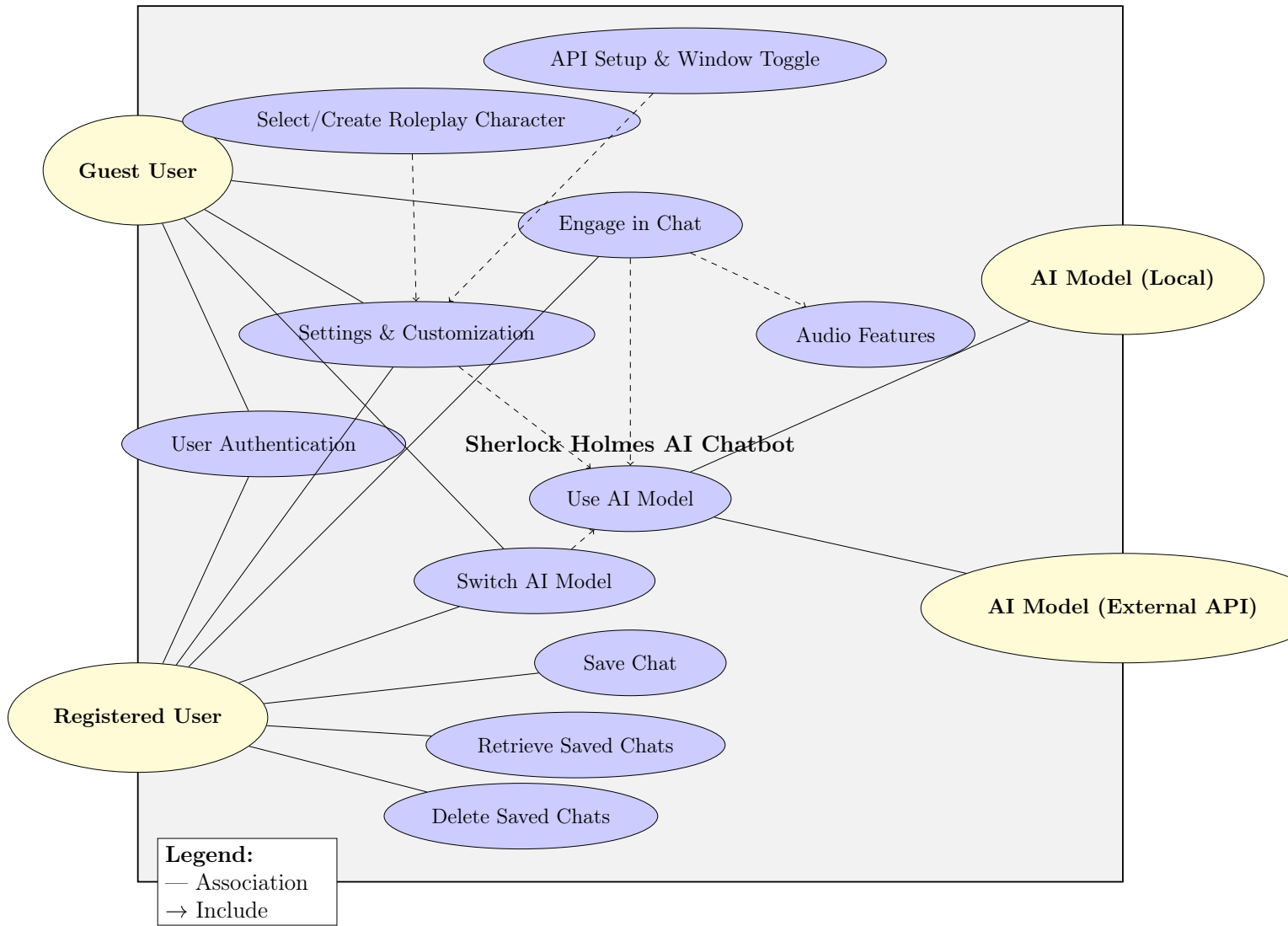


Figure 8: Use Case Diagram for Sherlock Holmes AI Chatbot Webapp

3.3.1.2 Non-Functional Requirements

Describes system reliability, scalability, security, usability, performance, accessibility and responsiveness.

1. Performance and Reliability

- **Response Time:** Generate AI responses within 10 seconds for the local model (may vary for external APIs).
- **Scalability:** Design the system architecture to support increases in user load and data volume for the future.

2. User Interface and Experience

- **Responsive Design:** Ensure full responsiveness across different desktop devices.

- **Intuitive Interface:** Design a clear, user-friendly chat interface resembling a text messaging app.
- **Consistency:** Maintain consistent design and functionality across all parts of the application.

3. Security and Privacy

- **Data Encryption:** Encrypt sensitive user data such as passwords.
- **Authentication Security:** Implement secure password hashing and storage practices.
- **Secure API Communication:** Conduct all external API communications over encrypted connections.
- **API Key Security:** Securely handle and store user-provided API keys on the client temporarily for online models.
- **Saved Chats:** Saved chats must be accessible exclusively to their respective authenticated creator.

4. Compatibility and Integration

- **Browser Compatibility:** Ensure the application functions correctly on major web browsers (Chrome, Firefox, Safari, Edge).
- **API Compatibility:** Design the system to easily integrate with various AI model APIs.

3.3.2 System Architecture Overview

The Sherlock Holmes AI Chatbot is designed as a modern web application where the system architecture is carefully crafted to provide a seamless, responsive, and secure user experience while efficiently managing complex backend processes including language model inferences and data management.

At its core, the application follows a client-server architecture, comprised of four main components: a dynamic frontend, a robust backend server, a relational database, and an AI model integration layer. The frontend, built with Vanilla JavaScript and styled with Tailwind CSS, offers an intuitive and responsive user interface. It features a chat interface with message history, controls for model selection, temperature adjustment, and other settings. This frontend communicates with the backend through RESTful APIs implemented using Flask, a lightweight Python web framework. The Flask backend serves as the central coordinator, managing user authentication, handling API requests, generating and orchestrating interactions between the frontend, database, and AI models. It also implements additional features such as text-to-speech functionality using the TTS

library, allowing for a more immersive user experience. The backend is designed to handle various routes and endpoints, including user management, chat operations, and AI model interactions. Data persistence is handled by an SQL database, accessed via SQLAlchemy ORM, which ensures efficient storage and retrieval of user information, chat histories, and other application data. This setup allows for scalable data management and quick access to user-specific information and chat logs. The character management system is implemented using a JSON-based storage (`characters.json`), it allows for the creation and management of various characters with distinct attributes, enhancing the roleplaying aspect of the chatbot. The language model component is particularly noteworthy, offering dual functionality for inference. It can leverage the locally hosted, fine-tuned LLaMA model that was trained in Section 3.2 for Sherlock Holmes emulation, managed by a custom `ModelManager` python class that utilizes `llama.cpp` Gerganov (2023) for efficient CPU-based inference. Alternatively, it can interface with external AI services via the OpenRouter API ope (2024), providing flexibility and scalability in AI processing capabilities. This dual approach allows the system to balance between local processing power and cloud-based scalability when needed.

User interactions in the frontend trigger API calls to the Flask backend, which processes these requests by interacting with the database or AI models (for inference or TTS generation) as needed. For AI responses, it either uses the local model via `ModelManager` or sends requests to the external API. Responses are then sent back to the frontend for display. This flow ensures real-time interaction while maintaining data consistency and security. The entire system is designed with deployment flexibility in mind, capable of running on server environments that support Python and SQLite, with security measures implemented throughout, including secure API key management for external services and user authentication protocols.

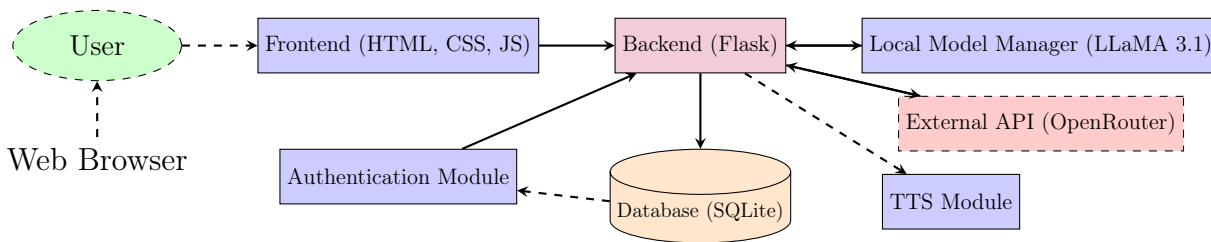


Figure 9: System Architecture Overview Diagram for Sherlock Holmes AI Chatbot

3.3.3 Backend Implementation

The backend of the Sherlock Holmes AI Chatbot is built using Flask, a lightweight WSGI web application framework in Python. This choice was made due to Flask’s flexibility, simplicity, and excellent integration with various extensions needed for the project. The backend is responsible for handling user requests, managing the AI model, processing chat

data, processing tts and interfacing with the database.

3.3.3.1 Flask Server Setup and Configuration

The `app.py` file serves as the entry point for the Flask application. It initializes the Flask app, sets up configuration options, and defines key components like the database and login manager. To manage configuration settings, the app uses environment variables, which are loaded via the `python-dotenv` package. This approach ensures that sensitive information like the SECRET KEY and database URLs are kept secure and can be easily changed without modifying the codebase. The server also initializes the custom `ModelManager` class for language inference and Text-to-Speech model at the start of the application to avoid reloading the model with each request. The application is designed to handle both synchronous and asynchronous tasks, such as the loading of AI models and the Text-to-Speech engine. These components are initialized at the start to ensure efficient performance.

Listing 9: Flask Server Initialization and Configuration Example Snippet

```
1 import os
2 from flask import Flask
3 from flask_sqlalchemy import SQLAlchemy
4 from flask_login import LoginManager
5 from dotenv import load_dotenv
6
7 load_dotenv()
8 app = Flask(__name__)
9 app.secret_key = os.getenv('SECRET_KEY', 'your_default_secret_key')
10 app.config['SQLALCHEMY_DATABASE_URI'] = os.getenv('DATABASE_URL', '
    sqlite:///sherlock_chatsv2.db')
11 db = SQLAlchemy(app)
12
13 login_manager = LoginManager()
14 login_manager.init_app(app)
15 login_manager.login_view = 'login'
16
17 model_manager = ModelManager('V5Sherlock8bit.gguf')
18 loading_thread = None
19 tts = TTS(model_name="tts_models/en/vctk/vits", progress_bar=False,
    gpu=False)
20
21 url = "https://openrouter.ai/api/v1/chat/completions" #
    OpenRouterAPI endpoint
```

3.3.3.2 API Design and Routing

The API design follows RESTful principles, providing clear and predictable endpoints for client-server communication. Each Flask routes associated with specific HTTP methods like GET, POST, and DELETE. The main API routes handle chat interactions, model inference, text-to-speech conversion, and user management.

The routing system is methodically organized to handle various functionalities. The main chat interface is served through `@app.route('/')`, providing the user interface. API endpoints for chat operations are encapsulated under `@app.route('/api/chats')`, ensuring a clean separation between frontend and backend operations. User authentication is managed through the `@app.route('/login')` and `@app.route('/register')` routes, while model management is handled by routes such as `@app.route('/load\_local\_model')`, which facilitates the loading and unloading of AI models.

Some of the Key routes include:

- **/get_response**: Processes chat requests, generating AI responses using either a local model or external API (OpenRouter). Includes error handling for model readiness and API issues.
- **/text_to_speech**: Converts text to speech using the VITS model, returning a WAV audio file. Handles various text inputs and includes error management.
- **/api/chats**: Manages chat records with GET (retrieve), POST (create), PUT (update), and DELETE (remove) methods. Ensures secure access to user-specific chat histories.
- **/login**, **/logout** , **/register** and **/api/user** : Handle user authentication and account creation, including credential validation and session management, manage user sessions and provide authentication status information.
- **/save_character** and **/get_characters**: Manage character profiles, allowing creation, updating, and retrieval of character data stored in JSON format.
- **/load_local_model**, **/unload_local_model**, **/check_model_status**: Control local AI model operations, including asynchronous loading, unloading, and status checking.
- **/local_model_inference**: Performs inference using the local AI model, handling input formatting and response generation.

Error handling is integrated into the routes to ensure users receive meaningful feedback when something goes wrong.

Listing 10: Simplified Example Snippet of Key Route `get_response`

```

1 @app.route('/get_response', methods=['POST'])
2 def get_response():
3     data = request.json #gets user message from frontend
4     if data.get('use_local_model', False):
5         # Local model handling if user wants to use local model
6         response = model_manager.inference(data['messages'][-1]['
            content'])
7         return jsonify({"choices": [{"message": {"content":
            response}}]})
8     else:
9         # External API handling for external Model Inference
10        api_key = request.headers.get('Authorization').split('
            Bearer ')[1]
11        headers = {"Authorization": f"Bearer {api_key}"}
12        response = requests.post(url, headers=headers, json=data)
13        return jsonify(response.json())

```

3.3.3.3 Language Model Management and Inference Handling

Local model management is mostly handled by the `ModelManager` class in `model_manager.py` which is responsible for tasks such as loading, unloading, and performing inference with the local model. Users have control over the model's state and can manually load or unload the model as needed, providing flexibility depending on their usage requirements so appropriate functions are provided. This is particularly useful for resource management, as users may choose to unload the model when it's not needed, freeing up system resources. The model is loaded based on conditional loading of GPU layers to enhance performance when GPU resources are available but still works even without it.

The local model is loaded asynchronously to avoid blocking the main application thread. Once loaded, the model's status is updated, and it can be used for inference requests. The inference process involves formatting the user's input, passing it to the model, and then returning the generated response. Local model inference is made possible through the use of the `llama_cpp` library Gerganov (2023), which provides Python bindings for the Llama model which my finetuned model is based on.

For external API calls, such as those to OpenRouter, the chatbot formats the input data according to the API's requirements and sends it to the endpoint. The response from the API is then processed and returned to the client in a structured format. When a request is made to the `/get_response` endpoint, the `ModelManager` handles the formatting of prompts and the execution of the inference.

Error handling is integrated into both the local and external inference processes. If the local model is not ready or if an external API call fails, the system returns a clear error message, ensuring that users are aware of any issues.

Listing 11: Simplified Example Local Model Loading and Inference Handling

```
1 def load_model(self):
2     self.status = 'loading'
3     self.model = Llama(
4         model_path=self.model_path,
5         n_ctx=2048,
6         n_gpu_layers=-1 if torch.cuda.is_available() else 0,
7         n_threads=8
8     )
9     self.status = 'ready'
10 def inference(self, messages):
11     if self.status != 'ready':
12         raise Exception("Model is not ready")
13     prompt = self.format_prompt(messages) # Formats messages into
14         format expected by the local model, adds delimiter tokens
15     response = self.model(prompt, max_tokens=100, stop=['<|eot_id|>'])
16     return response['choices'][0]['text'].strip()
```

3.3.3.4 Text-to-Speech Integration

The Text-to-Speech (TTS) functionality is implemented using the VITS model, which is integrated into the application via the TTS package. The TTS service converts text input into audio, which is then sent back to the client as a WAV file. The TTS model is loaded once during application startup to avoid reloading it with every request, which optimizes performance. The `text_to_speech` route handles incoming requests, processes the text, and returns the generated audio

Listing 12: Simplified Example TTS Handling

```
1 @app.route('/text_to_speech', methods=['POST'])
2 def text_to_speech():
3     data = request.json
4     text = data['text']
5     audio_bytes = io.BytesIO()
6     tts.tts_to_file(text=text, file_path=audio_bytes, speaker="p267",
7         speed=0.1, pitch=0.5)
8     audio_bytes.seek(0)
9     return send_file(audio_bytes, mimetype="audio/wav")
```

3.3.3.5 User Authentication and Session Management

User authentication and session management in webapp are implemented using the Flask-Login extension, which simplifies the handling of user sessions. The authentication process begins with user registration, where credentials, including a hashed password, are securely

stored in the database (detailed in the next section) using SQLAlchemy. During login, the provided password is compared with the stored hash using `werkzeug.security`, ensuring that only valid users can access the application. Once authenticated, Flask-Login manages the user's session, allowing them to remain logged in across multiple interactions without needing to re-authenticate. The `@login_required` decorator is used to restrict access to specific routes (such as `get_chats`), ensuring that only logged-in users can perform certain actions or view certain pages.

Listing 13: Simplified Example User Authentication and Restricted Access

```
1
2 @app.route('/login', methods=['POST'])
3 def login():
4     username = request.form.get('username')
5     password = request.form.get('password')
6     user = User.query.filter_by(username=username).first()
7     if user and check_password_hash(user.password_hash, password):
8         login_user(user)
9         return jsonify({"success": True, "message": "Login
10                        successful"}), 200
11     return jsonify({"success": False, "message": "Invalid username
12                   or password"}), 401
13
14 @app.route('/api/chats', methods=['GET'])
15 @login_required
16 def get_chats():
17     chats = Chat.query.filter_by(user_id=current_user.id).order_by(
18         Chat.created_at.desc()).all()
19     return jsonify([
20         {
21             "id": chat.id,
22             "title": chat.title,
23             "preview": chat.preview,
24             "created_at": chat.created_at.isoformat(),
25             "character": chat.character
26         } for chat in chats])
```

In these examples, the `@login_required` decorator ensures that only users who are logged in can retrieve their chat history or create new chats. If a non-authenticated user attempts to save a new chat, they are asked login first.

3.3.3.6 Custom Character backend

users to create and save custom characters for roleplaying purposes. This feature is implemented through dedicated API routes that manage character data stored in a JSON file (`characters.json`). Users can define new characters or update existing ones, specifying attributes such as name, personality traits, and role-playing scenarios.

When a new character is created or an existing one is updated, the application first loads the existing characters from the JSON file. It then checks if the character already exists—if it does, the character’s data is updated; if not, the new character is added to the list. The updated list is then saved back to the JSON file. This feature allows users to personalize their interactions with the chat bot, making the role-playing experience more immersive and tailored to their preferences.

Listing 14: Simplified Code Snipped Saving Custom Roleplay Character

```
1 def save_character():
2     new_character = request.json
3     characters = load_characters()
4     existing_char = next((char for char in characters if char['name
5         '] == new_character['name']), None)
6     if existing_char:
7         existing_char.update(new_character)
8     else:
9         characters.append(new_character)
10    with open(CHARACTERS_FILE, 'w') as f:
11        json.dump(characters, f, indent=2)
12    return jsonify(new_character), 201
```

3.3.4 Database Design

The webapplication utilizes a robust database system to manage user data, chat histories, and application state. It uses SQLAlchemy, a powerful SQL toolkit and Object-Relational Mapping (ORM) library for Python. This choice allows for efficient database operations and provides a high level of abstraction and its ability to work seamlessly with Flask making it easier to work with database objects in a Pythonic way.

In the application, SQLAlchemy is initialized in the `app.py` file, where the Flask app is configured (See Listing 9) to connect to the SQLite database. The `db` object is created as an instance of `SQLAlchemy`, and this object is used to define the database models (i.e., the tables and their relationships). Each model in the application, such as `User` and `Chat`, is defined as a Python class that inherits from `db.Model`. This class structure allows for easy definition of table columns, data types, and relationships between tables. This configuration allows the application to connect to different database systems (e.g., SQLite for development, PostgreSQL for production) by simply changing the `DATABASE_URL` environment variable. In my particular case, the line configures the app to connect to a SQLite database named `sherlock_chatsv2.db` by default

SQLAlchemy handles the creation of database tables through model definitions in the `models.py` file. Each model represents a table in the database, with class attributes corresponding to columns in the table. The `User` model represents registered users, while the `Chat` model stores individual chat sessions. The relationship between these models is

established through the `user_id` foreign key in the Chat model, creating a one-to-many relationship where one user can have multiple chats.

1. User

- **id**: Primary key, uniquely identifying each user.
- **username**: The username chosen by the user.
- **email**: The user's email address.
- **password_hash**: A hashed version of the user's password for secure authentication.

2. Chat

- **id**: Primary key, uniquely identifying each chat.
- **user_id**: Foreign key linking to the User entity, representing the user who initiated the chat.
- **title**: Title of the chat session.
- **preview**: A brief preview or summary of the chat content.
- **full_content**: The complete content of the chat session including the full system message, all user and assistant messages.
- **created_at**: Timestamp indicating when the chat was created.

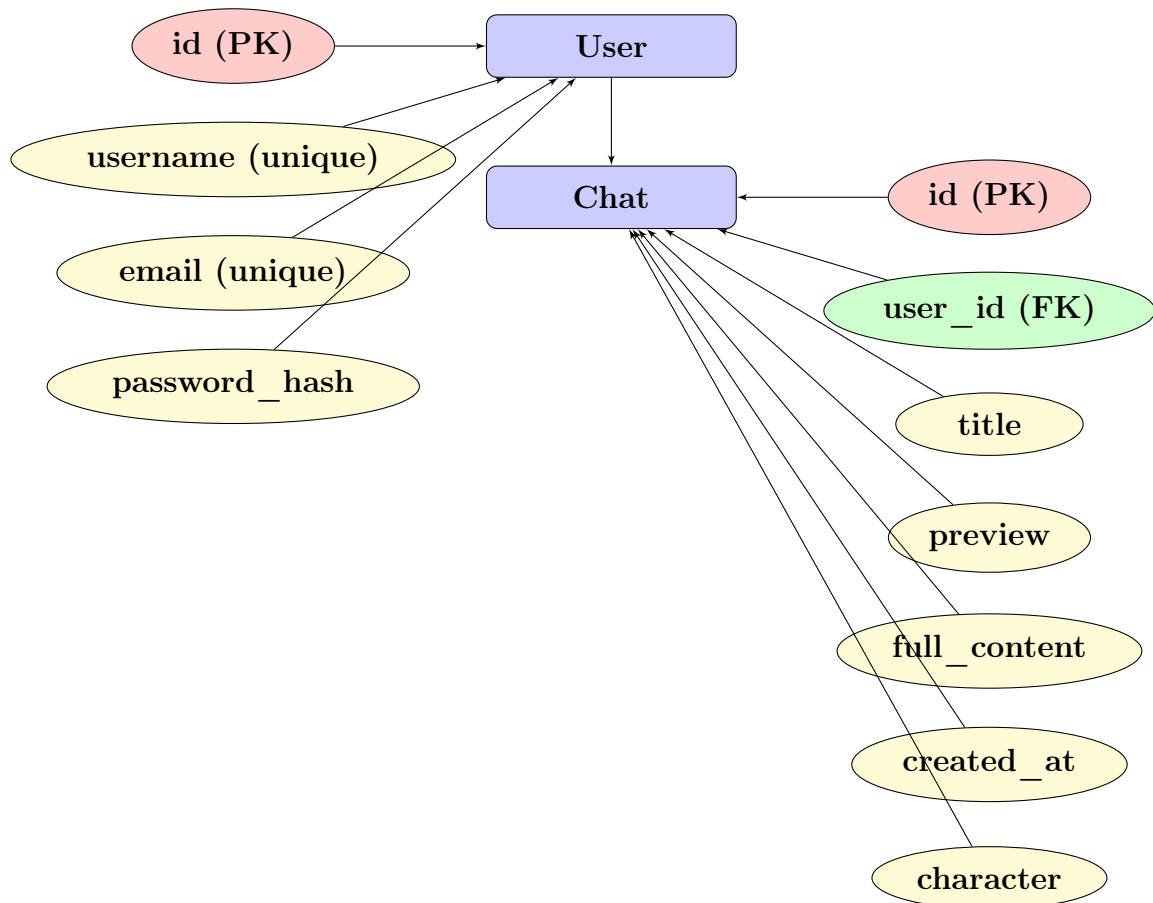


Figure 10: Entity-Relationship Diagram (ERD) for the User and Chat Models with Adjusted Layout

The database tables are created by running an initialization script (`init_db.py`), which calls `db.create_all()` to generate the tables based on the defined models. This ensures that all necessary tables are created before the application starts handling requests.

Listing 15: Database Initialization

```

1 def create_database():
2     with app.app_context():
3         db.create_all() # Create all tables based on the models
4         print("Database initialized successfully!")

```

Data retrieval and management in the application API routes are handled through various SQLAlchemy queries and functions. For example, retrieving all chats associated with a user and ordering by time created involves querying the Chat table using the `user_id` foreign key:

Listing 16: SQLAlchemy query for chat retrieval within the `'/api/chats'` route

```

1 @app.route('/api/chats', methods=['GET'])
2 @login_required
3 def get_chats():

```

```
4     chats = Chat.query.filter_by(user_id=current_user.id).order_by(  
        Chat.created_at.desc()).all()  
5     return jsonify([  
6         "id": chat.id,  
7         "title": chat.title,  
8         "preview": chat.preview,  
9         "created_at": chat.created_at.isoformat(),  
10        "character": chat.character  
11    ] for chat in chats])
```

These operations are wrapped in try-except blocks to handle any errors gracefully and ensure that the database remains consistent. Deleting records is also straightforward with SQLAlchemy, ensuring that all associated data is removed from the database

3.3.5 Front-end Design and Implementation

3.3.5.1 Overall Design Philosophy and UX Goals

The design of the Sherlock Holmes Chatbot interface. employs a modern, clean design that prioritizes functionality and user experience. A deep navy blue color scheme evokes Holmes' iconic overcoat, while the mix of serif headings and sans-serif body text balances Victorian-era aesthetics with contemporary readability. The detective's silhouette in the header instantly establishes the theme, complemented by UI elements featuring soft shadows and rounded corners. The primary goal is to ensure users can seamlessly interact with the Sherlock Holmes model, switch between local and external models, and manage their role-play sessions securely and without any confusion. By employing a modern chat interface, familiar navigation patterns, and responsive design elements, the chat-bot offers a clean, accessible, and highly interactive user experience.

HTML5 provides the structural framework for the interface. It ensures a well-organized, semantic layout that is easy to maintain and expand. JavaScript handles most of the dynamic functionality, such as model switching, sending messages, managing API interactions and handling audio playback for TTS . It plays an important role in processing user input and updating the UI in real time. Tailwind CSS is used extensively for styling, ensuring that the interface remains clean, modern, and responsive across different screen sizes. It provides a utility-first approach that allows for rapid development and easy customization of the visual elements.

3.3.5.2 Layout of the Interface

The overall interface of the main page is structured around three key sections allowing for a clear separation between the chat interaction, settings, and user-related features: :

1. **Main Chat Window:**



Sherlock Holmes: "Missed him? Watson, we didn't *miss* him. We were *not* pursuing him. Observe," I said, gesturing with a sweep of my hand towards the muddy bank where footprints imprinted the soft earth. "The escape route is clear. He's not a man of haste, not in his choice of path. He knew the terrain, and he knew precisely where to tread." I knelt, examining the prints carefully. "Look at the pressure of each step. Notice the angle of the heel. This was no hurried flight, Watson. This was a calculated retreat." I looked up at you, my eyes sharp and intent. "Tell me, Watson, what do you deduce from the state of his tracks?" ▶

Dr. Watson: I dont know you tell me

Sherlock Holmes: "Elementary, my dear Watson! The mud is thickest here, yet the prints are remarkably clear. He didn't simply walk through the mud. He *waded* through it. Deliberately, methodically. Why?" I paused, letting the question hang in the air. "He didn't want to leave a clean trail. He wanted to obscure his movements, yet he still needed to be able to navigate the terrain easily." My gaze drifted back to the river, a hint of a smile playing on my lips. "A man who knows the area well, a man confident in his ability to move unseen. A man, I suspect, who has done this before." I rose, brushing the mud from my coat. "Come, Watson. We have much to consider. The footprints tell us a great deal, but they are only the beginning." ▶

Dr. Watson: where should we go now

Sherlock Holmes: "We go where the thread leads, Watson. Not blindly, but with a discerning eye. The riverbank is important, yes, but it's a stage, a path he chose to walk. We need to understand the play, the motivations, the context." I tapped my chin, considering. "The prints suggest familiarity with this stretch of the river. That means local knowledge. Perhaps a place frequented by our elusive gentleman, a place where he might be expected." I looked at you expectantly. "Do you recall any mention of local pubs, frequented by those who might have a connection to our case?" My gaze fixed on the swirling currents of the river, my mind already racing through possibilities. "Time is of the essence, Watson. We must act swiftly, but with purpose." ▶

Sherlock Holmes: 🤔 Contemplating...

Start New

Save Chat

Type your message here...

Send

```
Sending to API: {
  "model": "google/gemma-2-9b-it:free",
  "messages": [
    {
```

Settings

Mustafa Mehmood ▾

Model Source:

OpenRouter API

Access various AI models



Local Fine-tuned Model

Use your custom model



Local Model Status:

Not Ready

Select Model:

Google Gemma 2 9B IT (Free) ▾

Confirm

API Key:

Enter your API key

Save Key

Clear

Context:

A suspect has just fled the crime scene. Sherlock and Watson are along the river side.

Character:

Dr. Watson ▾

Custom Character

Temperature:

0.7

API Window Toggle

Saved Chats

Help me with the missing ring...

The missing ring, a most intriguing case, Mustafa Mehmood. I presume? (pausing to glance at you) I must say, I'm delighted to be working with the creator of yours truly. (smiling) Now, let us proceed with the facts. Tell me, what is the nature of this ring and who is its rightful owner? As I sit here, I notice the surroundings. Your office, I assume? (looking around) A tidy space, well-organized,

9/3/2024, 11:51:41 PM

Where is the missing ring?...

(sitting in my armchair, puffing on my pipe) Ah, the ring. A simple enough case, Watson. Or so it would seem. The owner, Lady Emily, claims it was stolen from her bedroom. The only clue is a suspicious character seen lurking around the estate on the night of the theft. (leaning forward) Now, let us examine the facts. Lady Emily's maid, Jenkins, reported that the ring was last seen on Lady Emily's finger during dinner. However,

9/3/2024, 11:50:52 PM

Figure 11: Vertically extended layout of the main page comprising three main sections: Main Chat Window, Settings panel, and the Saved Chat section. OpenRouter Model, Gemma is selected, and the user Mustafa is role-playing as Dr. Watson.

- The central section of the interface is the chat window, where the conversation takes place between the user (the role-played character) and Sherlock Holmes (For example in 11 Dr. Watson) and Sherlock Holmes. Each message in the chat history window is styled with a text message bubble effect, creating a clear distinction between user input and model-generated responses. The messages are aligned to either the left or right, depending on whether it's a user message or Sherlock's reply, mimicking typical chat applications.
- Next to each message is a small play button, allowing users to listen to the message via a text-to-speech feature. When clicked, the button animates while the audio is being generated and changes to a pause button during playback. This feedback helps users know when the audio is ready and currently playing.
- Below the chat history is the message input field where users type their messages as the character they are role-playing. This area includes a text box for typing and a Send button, which submits the user's input, processes it and passes it to the backend for generating inference.
- The Save Chat button lets users store their conversation in the database. However, this feature is only available to logged-in users. If a user attempts to save a chat without being logged in, they will be prompted to log in first. The Start New Chat button clears the current conversation and resets the interface for a new session with the AI. This button allows users to begin a fresh conversation quickly and easily.
- After the user sends a message, an animation labeled "Sherlock Holmes is contemplating..." appears just beneath the chat history, mimicking the familiar typing indicator in messaging apps. This gives users real-time feedback that the model is generating a response and a message will be returned within a couple of seconds.
- A toggle-able API message window inspection feature is available under the input area. This option can be enabled via the settings panel, allowing advanced users to inspect the API interaction, which shows information such as the model in use, inference configuration, the full message being sent, and other details for debugging.

2. Settings Panel:

- Located to the right of the chat window, the Settings Panel provides all the customization and model control features. Users can select different AI models, set the context, and manage role-play characters. Its contents adapt dynamically based on the selected Model Source.

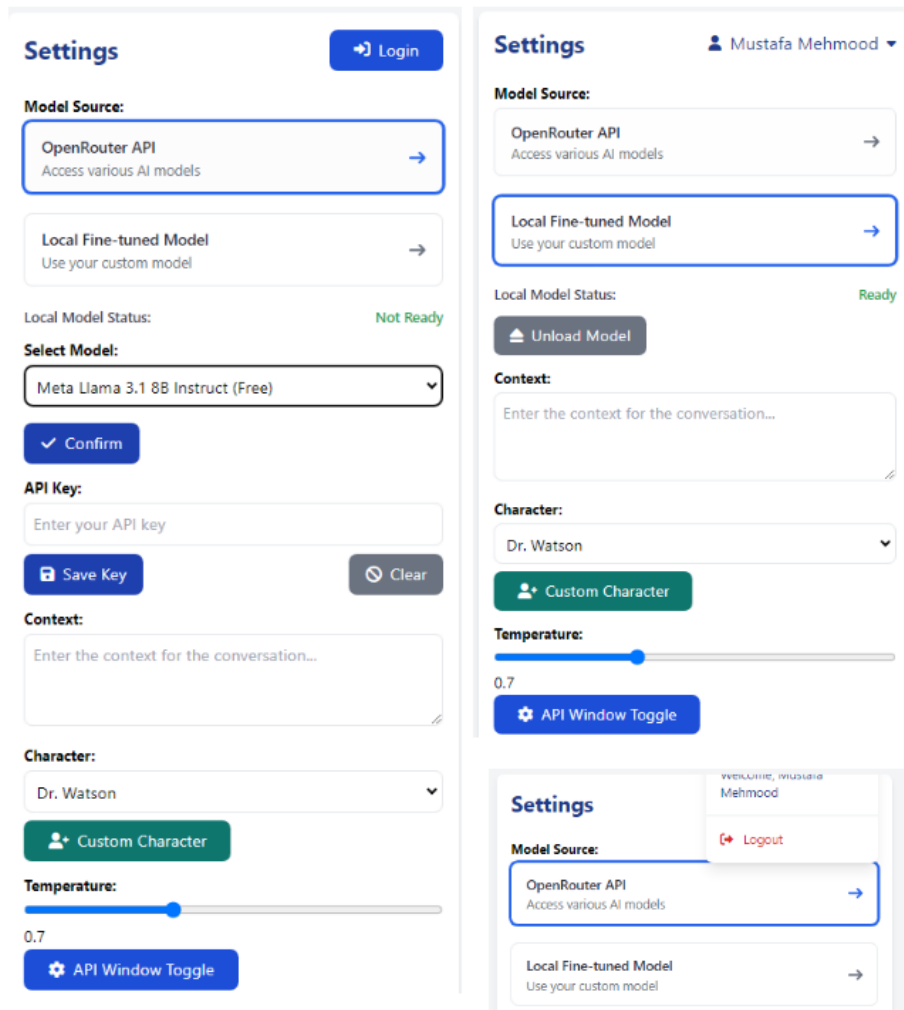


Figure 12: Dynamic UI Showcase for Settings Panel

- **Model Source Selection:** Users can choose between an OpenRouter API for external AI models and a Local Fine-tuned Model. This selection dynamically affects the UI components displayed. When the OpenRouter API is selected, the interface reveals additional options: a drop-down menu to choose specific models (e.g., Google Gemma, Meta Llama), an API Key input field for entering and saving OpenRouter API credentials, and Save Key and Clear buttons for managing API credentials. Conversely, selecting the Local Fine-tuned Model hides OpenRouter-related components and instead displays a Local Model Status indicator (showing "Ready" or "Not Ready") and an Unload Model button when the local model is loaded. This dynamic UI approach ensures users see only relevant options based on their chosen model source, streamlining the interface.
- The optional context text input allows users to provide context for the chat such as the setting or case background, tailoring the behavior of the model.
- Users can select between predefined characters (e.g., Dr. Watson, Mrs.Hudson)

to role-play as via the dropdown menu. If users want to create their own custom characters or just roleplay as themselves, they can click on the Custom Character button to open the form for character creation detailed later.

- A slider is available to adjust the randomness (temperature) of the model responses. Lower temperatures produce more deterministic, factual outputs, while higher temperatures introduce more creative, diverse replies.
- A button at the bottom of the settings panel allows users to toggle the API message inspection window on or off in the chat area, enabling or disabling the display of API interaction details.
- The user's profile information and authentication options are located at the top right of the Settings Panel. When a user is not logged in, the Login button is prominently displayed. Upon clicking, users are redirected to the login page. When authenticated, the user's name appears (e.g., "Mustafa Mehmood") with a dropdown menu allowing them to access the Logout option. The availability of certain features, such as managing API keys, saving chats, and viewing the Saved Chats Section, is dependent on whether the user is authenticated.

3. Saved Chats Section:

- At the bottom (footer) of the interface is the Saved Chats Section, where logged in users can access and manage their previously saved conversations when they scroll down. Each chat is displayed with a brief title, preview, and timestamp for easy identification. There is a cross icon which lets the user delete chats after a confirmation modal dialogue popup.

Other than the main page, the interface also involves other elements such as the Login and Register pages and the Custom Character Creation form:

The figure displays two user authentication pages for the 'Sherlock Holmes Chatbot'. Both pages feature a dark blue header with the chatbot's logo and name. The top page is the registration form, which includes input fields for 'Username', 'Email', and 'Password', followed by a blue 'Register' button. Below the button is a link that says 'Already have an account? Login'. The bottom page is the login form, which includes input fields for 'Username' and 'Password', followed by a blue 'Login' button. Below the button is a link that says 'Don't have an account? Register'.

Figure 13: User Authentication Pages

3.3.5.3 User Authentication Pages

The pages are styled for consistency with the rest of the interface and the main webpage. The registration page offers fields for a username, email, and password, enabling account creation and access to features like chat saving. Upon registration, users are redirected to the login page, which has a simple two-field form (username and password) for authentication. Upon successful login the users are redirected to the main page, the chat interface. Both the login and registration pages provide options to navigate between each other.

Custom Character Creation Screen

Create Custom Character

Name (required):
Mustafa Mehmood

Description (required):
A CSAI Masters Student at UoN.

Relationship to Sherlock:
Creator

Traits:
Helpful ×
Add a trait +

Speaking Style:
Makes a lot of spelling and grammatical mistakes.

Sherlock's Approach:
Sherlock treats them with respect as they are the creator.

Cancel Save Character

Figure 14: Custom Role-play Character Creation Modal Form

- **Modal Form Design:** When users opt to create a custom character, a modal form is triggered. This form allows for the customization of various character attributes, enabling a more immersive roleplay experience.
- **Character Name and Description:** The form starts with fields for the character's name and a brief description, allowing users to create personalized characters with rich backstories.
- **Relationship to Sherlock(optional):** Users can define the character's relationship with Sherlock (e.g., "friend," "rival," or "creator"), which affects how the chatbot interprets and responds to the character during interactions.
- **Traits and Speaking Style(optional):** Users can add traits that define the character's personality, such as "helpful" or "curious." In addition, the speaking style can

be customized, allowing users to adjust how the character phrases responses (e.g., “formal,” “friendly,” or with grammatical quirks).

- **Sherlock’s Approach(optional):** The final field allows users to define how Sherlock treats this character, providing another layer of customization to influence the chatbot’s responses during roleplay.
- **Save and Manage Custom Characters:** Once the character is defined, the user can save it, and it becomes available for selection in the "Character" dropdown in the settings panel.

3.3.5.4 Front-end Implementation of Key Operations

The script.js file is structured to handle all the interactive elements of the frontend, including chat message handling, asynchronous operations, and UI updates. The script is organized into functions that manage specific tasks, ensuring that the code is modular and easy to maintain. AJAX calls are made using the Fetch API to communicate with the backend without reloading the page. These calls handle tasks such as sending messages, retrieving AI responses, performing TTS conversions, managing custom characters, handling user authentication and retrieving saved chats. Each call includes error handling to manage scenarios like network failures or server errors.

How sending, receiving and updating messages works?

Begins with model selection, API setup in case of Open-router, loading model incase of local model, building and sending inference requests, receiving messages and finally updating the Chat history window works?

The settings panel UI presents radio buttons for users to select their preferred model source. When a user changes the selection, an event listener triggers the `updateUIForModelSource()` function which shows or hides relevant UI elements based on the selected model source.. The `currentModelSource` variable is updated to reflect the user’s choice.

Listing 17: Simplified Example:Model Source Event Listener

```
1 modelSourceRadios.forEach((radio) => {
2   radio.addEventListener('change', (e) => {
3     currentModelSource = e.target.value;
4     updateUIForModelSource();
5     if (currentModelSource === 'local') {
6       checkAndLoadLocalModel();
7     }
8   });
9 });
```

If the local model is selected, it triggers `checkAndLoadLocalModel()` function which sends a request to the backend to start loading the local fine tuned model if it is not loaded.

Listing 18: Simplified Example:Sending Request to the backend to load model

```

1 function checkAndLoadLocalModel() {
2     const statusElement = document.getElementById('local-model-
      status');
3     if (statusElement.textContent === 'Not Loaded') {
4         fetch('/load_local_model', { method: 'POST' })
5             .then(response => response.json())
6             .then(data => {
7                 if (data.message === "Model loading started") {
8                     checkLocalModelStatus();}})
9             .catch(error => console.error('Error loading model:', error
      ));
10    }
11 }

```

When OpenRouter is selected, users have access to multiple model options.(like Meta Llama, Google Gemma, Mistral, etc.) from a drop down menu. The choice is saved to the local storage. Users can input their own API key for openrouter models for flexibility. API keys are also saved using the `localStorage.setItem()` method when the user clicks the "Save Key" button. The API key and the particular model to use is later retrieved from `localStorage` whenever a request is sent to OpenRouter

Listing 19: Simplified Example:Handeling model selection and API key for Openrouter inference

```

1 document.getElementById('save-model-btn').addEventListener('click',
      function() {
2     const model = document.getElementById('model-select').value;
3     localStorage.setItem('SelectedModel', model);
4     alert('Model saved successfully!');
5 });
6
7 document.getElementById('save-api-key btn').addEventListener('
      click', function() {
8     const apiKey = document.getElementById('api-key').value;
9     localStorage.setItem('APIKey', apiKey);
10    alert('API Key saved successfully!');
11 });

```

When sending a message, the application uses the selected model from the `currentModelSource` variable and calls the appropriate functions depending on the source.

Listing 20: Simplified Example:Calling appropriate functions for inference based on model source

```

1 function sendMessage() {
2     // ... other code ...
3     if (currentModelSource === 'openrouter') {

```

```
4         sendOpenRouterRequest(userMessage, temperature);
5     } else {
6         sendLocalModelRequest(userMessage, temperature);
7     }
8 }
```

For Open-Router requests, it includes the selected model in the API call and the API key is fetched and used to authorize and the request:

Listing 21: Simplified Example:How Openrouter model inference request is made to the backend

```
1 function sendOpenRouterRequest(temperature) {
2     const apiKey = localStorage.getItem('APIKey');
3     const model = localStorage.getItem('SelectedModel') //
      Retrieves API key from local storage
4     const apiRequestBody = {
5         model: model,
6         messages: currentChat,
7         temperature: temperature,
8     };
9     fetch('/get_response', {
10         method: 'POST',
11         headers: {
12             'Content-Type': 'application/json',
13             'Authorization': 'Bearer ${apiKey}', // API key in
              request header
14         },
15         body: JSON.stringify(apiRequestBody),
16     }).then(response => response.json()).then(data =>
17         handleModelResponse(data))
18     .catch(handleModelError);
19 }
```

The selected model within open router models list is included in the request body sent to the backend in case of open router but for the local model, a different endpoint ('/local_model_inference') is used, implicitly telling the backend to use the local model via the sendLocalModelRequest() function.

Listing 22: Simplified Example:How local model inference request is made to the backend

```
1 function sendLocalModelRequest(temperature) {
2     const apiRequestBody = {
3         messages: currentChat,
4         temperature: temperature,
5     };
6     apiMessage.textContent = 'Sending to Local Model: ${JSON.
      stringify(apiRequestBody, null, 2)}';
```

```

7     document.getElementById('loading-indicator').classList.toggle('
      hidden');
8     fetch('/local_model_inference', {
9         method: 'POST',
10        headers: {
11            'Content-Type': 'application/json',
12        },
13        body: JSON.stringify(apiRequestBody),})
14    .then(response => response.json())
15    .then(data => {
16        handleModelResponse(data);})
17    .catch(handleModelError);
18 }

```

The entire chat history is sent with each request to maintain context. This ensures that the model has access to previous messages to generate coherent responses. The array `currentChat` stores all messages (system, user, assistant). When the first message is sent in the beginning of a new chat or when role-play character is changed or the context input is updated in the settings panel, the frontend captures generates a system message using `generateSystemMessage()` and makes sure the system message is always included as the first message in the `currentChat` array.. This function **creates a context aware prompt**, incorporating the **base Sherlock Holmes character description** instruction to act as Sherlock, any **additional context** provided by the user and specific **details about the character Sherlock is conversing with** (if available).

Listing 23: Simplified Example:How system message is generated

```

1 function generateSystemMessage(context, character) {
2     let message = 'You are Sherlock Holmes, the famous detective
      known for .....'; #Half only for demonstration
3
4     if (context) {message += ' Context: ${context}';}
5     message += ' You are currently in a conversation with ';
6     const currentChar = customCharacters.find(char => char.name ===
      character);
7     if (currentChar) {
8         message += `${currentChar.name}. ${currentChar.description}
          ;
9         if (currentChar.relationship) message += 'Their
          relationship to you is ${currentChar.relationship}. ';
10        if (currentChar.traits && currentChar.traits.length > 0)
          message += 'They have the following traits: ${
          currentChar.traits.join(', ')}. ';
11        if (currentChar.speakingStyle) message += 'Their speaking
          style: ${currentChar.speakingStyle}. ';
12        if (currentChar.sherlockApproach) message += 'Your approach

```

```

13         to them: ${currentChar.sherlockApproach}. ‘;
        message += ‘Adjust your tone and manner of speaking
            accordingly.’;
14     } else {
15         message += ‘an unknown individual.’;
16     }
17
18     return message;
19 }

```

For handling responses from both OpenRouter API and local model, the application uses a common function called `handleModelResponse()`. The function first hides the loading indicator ("Sherlock is contemplating."), signaling that the response has been received, then extracts the actual text, calls a function (`addMessageToChat()`) to update the UI and also adds the new response to the `currentChat`. Finally, it sets up the audio button for text-to-speech functionality.

Listing 24: Simplified Example:How returned model responses are handled

```

1 function handleModelResponse(data) {
2     document.getElementById('loading-indicator').classList.toggle('
        hidden');
3     const sherlockResponse = data.choices ? data.choices[0].message
        .content : data.response;
4     addMessageToChat('assistant', sherlockResponse);
5     currentChat.push({ role: "assistant", content: sherlockResponse
        });
6
7     const lastMessage = chatHistory.lastElementChild;
8     const audioButton = lastMessage.querySelector('.audio-button');
9     audioButton.className = 'audio-button ready';
10    audioButton.innerHTML = 'X'; //a play button latex cant render
11 }

```

Messages are dynamically added to the chat interface using the `addMessageToChat` function. This function is called whenever a new message needs to be displayed, whether it's a user message or a response from the model. This function creates HTML elements for each message and appends them to the chat window.

Listing 25: Simplified Example:Cases when the `addMessageToChat` function is called

```

1     \\when a user sends a message
2     addMessageToChat('user', userMessage);
3     \\when a model response is received
4     addMessageToChat('assistant', sherlockResponse);
5     \\When loading saved chats
6     currentChat.forEach(message => {
7         if (message.role !== 'system') {

```

```
8         addMessageToChat(message.role, message.content);  
9     }  
10 });
```

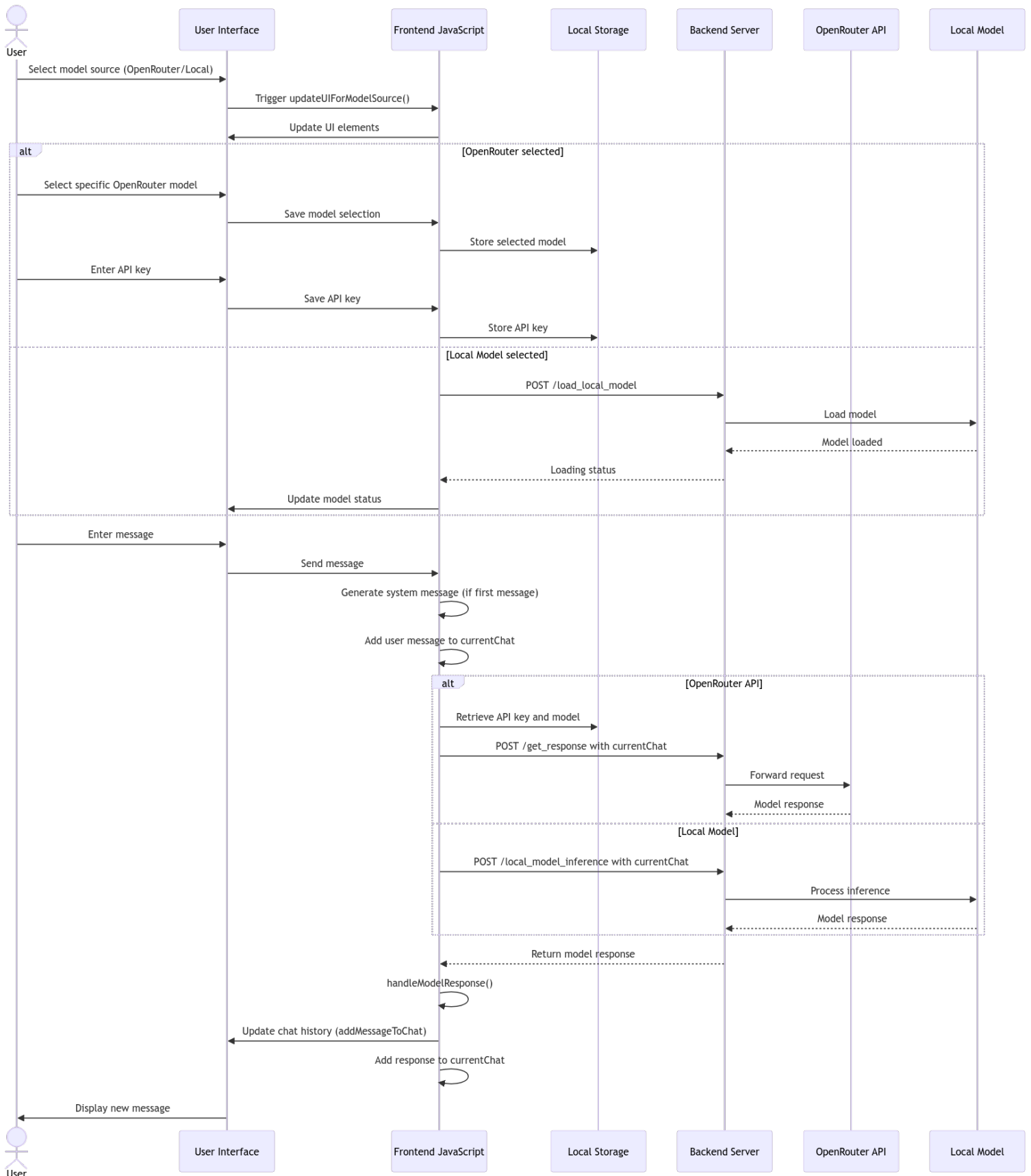



Figure 15: Sequence Diagram: Model Switching, Sending requests and receiving inferences

How does the audio playback feature works?

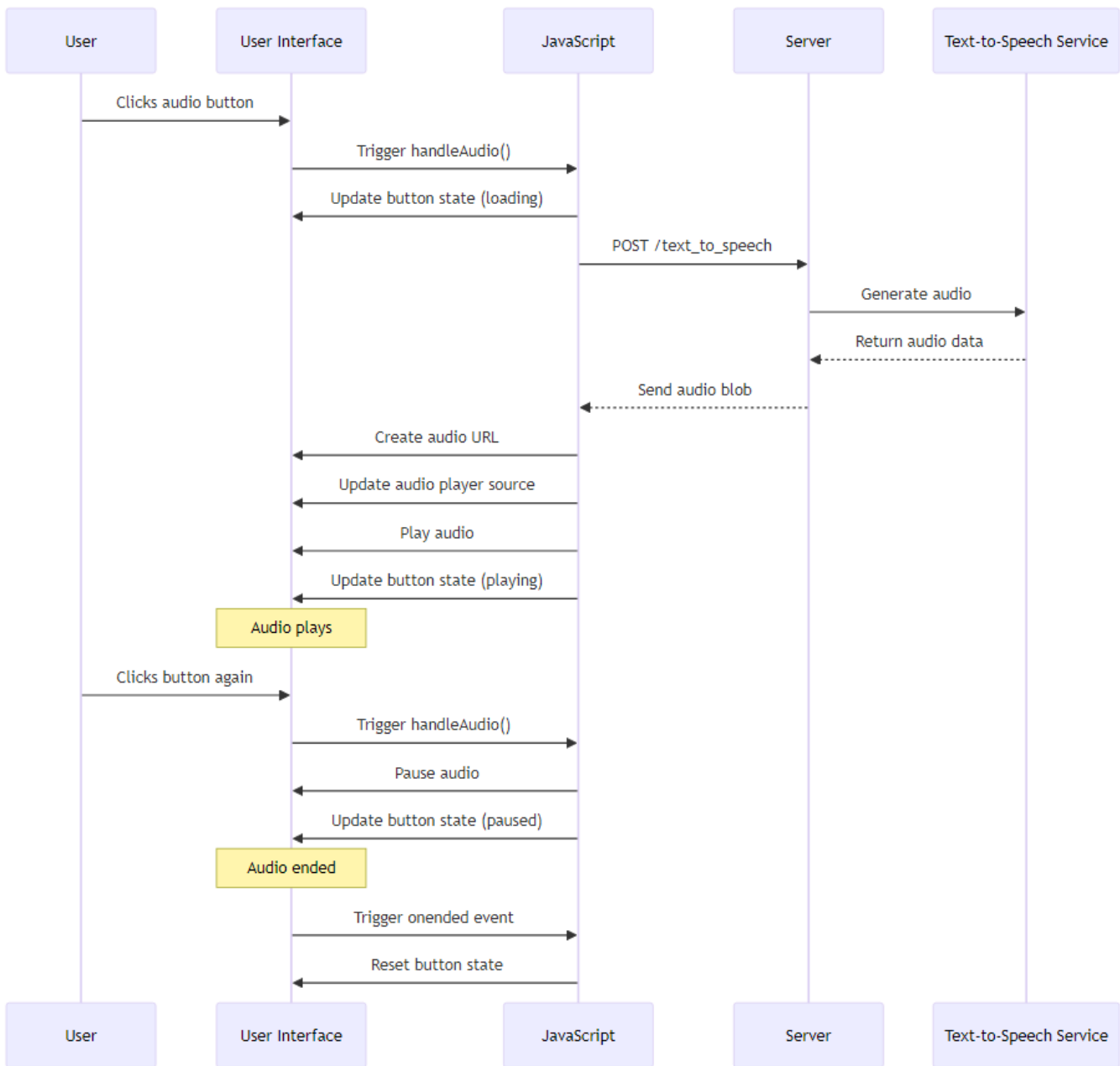


Figure 16: Sequence Diagram for Audio Playback

When a model response is added to the chat an audio button is created and appended to the response in the text bubble, when this is clicked it triggers the `handleAudio()` function. This function manages the state of the audio button and triggers audio fetching when needed. On the server, the text-to-speech conversion is handled by a Flask route which uses the TTS library package to generate audio from the provided text (Detailed in Back-end Implementation). The client receives the audio data, creates a playable URL, and starts playing the audio. The user can now control playback using the audio button. Clicking it while playing will pause the audio, and clicking again will resume. When the audio finishes playing, the button state is reset. When navigating away from the page or loading a new chat, any existing audio URLs is revoked to free up memory

Listing 26: Simplified Example: Some important frontend JS functions for processing audio playback

```
1      button.textContent = 'XX'; // Loading indicator not rendered by
      latex
2      button.disabled = true;
3
4      fetch('/text_to_speech', {
5          method: 'POST',
6          body: JSON.stringify({text: text}),
7      })
8      .then(response => response.blob())
9      .then(blob => {
10         audioPlayer.src = URL.createObjectURL(blob);
11         audioPlayer.play();
12         button.textContent = 'X'; //Play button not rendered by
            latex
13         button.disabled = false;
14     })
15     .catch(error => {
16         console.error('Audio fetch error:', error);
17         button.textContent = 'X'; //Play button not rendered by
            latex
18         button.disabled = false;
19     });
20 }
21
22 function handleAudio(text, button) {
23     const audioPlayer = document.getElementById('audio-player');
24
25     if (!audioPlayer.src) {
26         fetchAudio(text, button, audioPlayer);
27     } else if (audioPlayer.paused) {
28         audioPlayer.play();
29         button.textContent = 'Y'; //Pause button not rendered by
            latex
30     } else {
31         audioPlayer.pause();
32         button.textContent = 'X'; //Play button not rendered by
            latex
33     }
34 }
```

How populating the character list and setting up custom characters works?

The application provides a modal form for creating custom characters to role play as which is triggered by clicking the custom character creation button in the settings panel. They fill out the form with details about the character. On submission, the character

data is sent to the server using the `saveCharacter()` function and saved. When starting a chat, The frontend fetches all saved characters and populates the character selection drop-down. When a user selects a character and starts a new conversation, the character's details along with the context if present are incorporated into the system message sent to the model and included in the `currentChat` array as the first message. This allows the model to adjust its responses based on the selected character, increasing the interactivity of role-play experience.

Listing 27: Simplified Example:saveCharacter function in Javascript

```
1 function saveCharacter(character) {  
2     fetch('/save_character', {  
3         method: 'POST',  
4         headers: {  
5             'Content-Type': 'application/json',  
6         },  
7         body: JSON.stringify(character)  
8     })  
9     .then(response => response.json())  
10    .then(data => {  
11        alert('Character saved successfully!');  
12        loadCharacters(); // Reload character list after saving  
13        //rest of the code  
14    })  
15 }
```

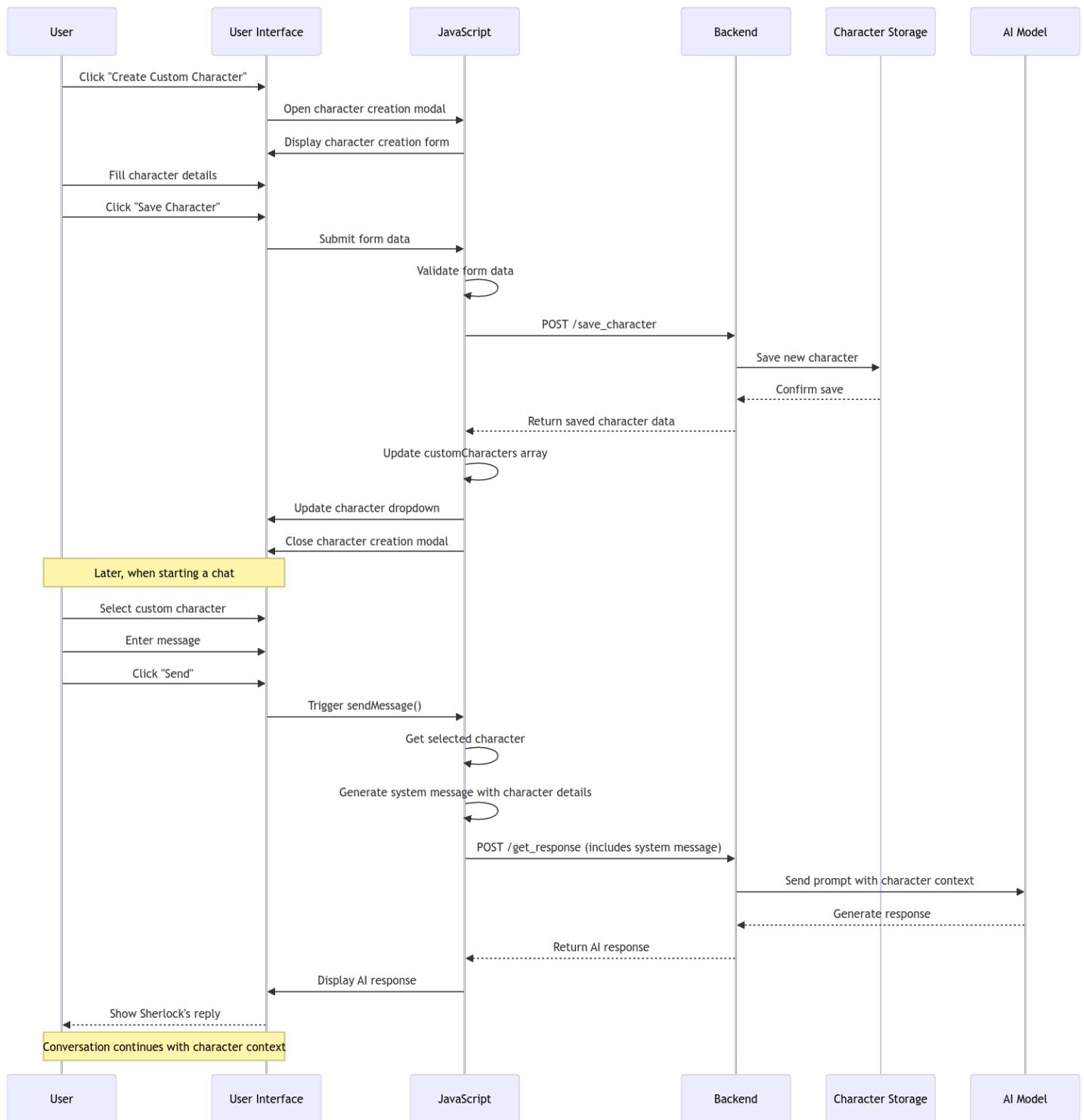


Figure 17: Sequence Diagram: Populating custom characters, setting up custom characters and incorporating into System Message

How does the user registration, login and saving chat work?

Users register or log in through the provided webpages in the front end (register.html, login.html). These can be easily navigated to via the main page from the settings panel. On the register page the user submits a form with their registration details (username,

email, password). The form is handled by the backend via the /register route in app.py (Detailed in the API routes section of Backed Implementation) The server checks if the username or email already exists in the database. If the details are unique, a new user account is created in the database, with the password being hashed for security and the user is redirected to the login screen. When existing users enter their credentials, the frontend sends these credentials to the server for verification.(User Model/Table detailed in the Database Design section) The server checks the provided username and password against the database and if they are correct, a session is created for the user, marking them as logged in after which they are redirected to the main chat interface with the updated UI (e.g., showing logout option, displaying username).This session allows the user to access protected routes and features, such as saving chats. When the save option is selected, the frontend collects the current chat data (currentChat Array) which includes system message and all the user and assistant message. It then sends a POST request to the server with the chat data. This includes the chat title, preview, full content, and character information. (Chat Model/Table detailed in the Database Design section)

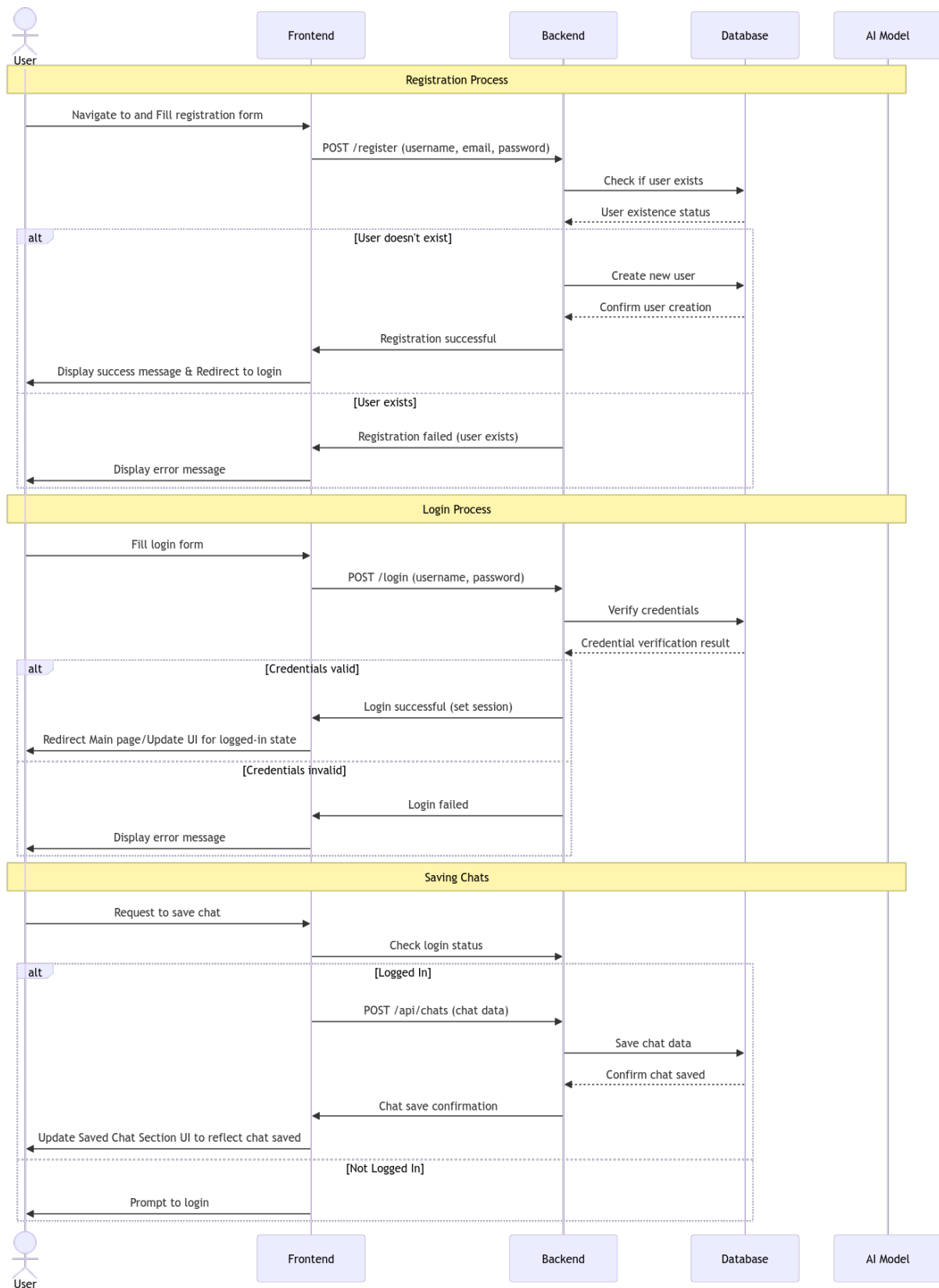


Figure 18: Sequence Diagram:Registration, Login and Saving

The frontend interacts send requests to the backend routes for user authentication and chat management via the following functions:

Listing 28: Simplified Example: Javascript Frontend Authentication and saving chats

```

1 function login(username, password) {
2     fetch('/login', {
3         method: 'POST',
4         headers: { 'Content-Type': 'application/x-www-form-
5             urlencoded' },
6         body: 'username=${encodeURIComponent(username)}&password=${
7             encodeURIComponent(password)}'
8     })
9     .then(response => response.json())
10    .then(data => {
11        if (data.success) {
12            checkLoginStatus();
13        } else {
14            alert(data.message);
15        }
16    });}
17
18 function saveCurrentChat() {
19     if (currentChat.length === 0) return;
20
21     const chatData = {
22         title: currentChat[0].content.substring(0, 30) + "...",
23         preview: currentChat[1] ? currentChat[1].content :
24             currentChat[0].content,
25         full_content: JSON.stringify(currentChat),
26         character: currentCharacter
27     };
28
29     fetch('/api/chats', {
30         method: 'POST',
31         headers: { 'Content-Type': 'application/json' },
32         body: JSON.stringify(chatData),
33     })
34     .then(response => response.json())
35     .then(data => {
36         alert("Chat saved successfully!");
37         fetchSavedChats();
38     });}

```

How the chat accessing and loading system works?

When a user logs in, the frontend fetches all their saved chats from the backend by making a call "fetch('/api/chats')" to the backend. This then triggers a call to the `updateSavedChatsList()` function which updates and displays all the retrieved chats (the preview, the title, and the timestamps) in the Saved Chats section of the main page. When the user selects a specific chat, the frontend send a fetch request to the backend route `/api/chats/${chatId}` along with the chatid via the `loadChat()` function. The full

content of the chat is parsed from the received JSON and set as the current chat. Each message in the loaded chat is displayed in the chat history and the character associated with the chat is set automatically.

Listing 29: Simplified Example:loadChat() function of the front end responsible for loading chats.

```
1 function loadChat(chatId) {
2     // 1. Fetch chat data from the server
3     const response = await fetch(`/api/chats/${chatId}`);
4     const chat = await response.json();
5
6     // 2. Clear the current chat display
7     clearChat();
8
9     // 3. Set the character for the loaded chat
10    currentCharacter = chat.character || 'Dr. Watson';
11    characterSelect.value = currentCharacter;
12
13    // 4. Parse and set the chat content
14    currentChat = JSON.parse(chat.full_content);
15
16    // 5. Display chat messages
17    currentChat.forEach(message => {
18        if (message.role !== 'system') {
19            addMessageToChat(message.role, message.content);
20        }
21    });
22
23    // 6. Update chat state
24    currentChatId = chatId;
25    isNewChat = false;
26 }
```

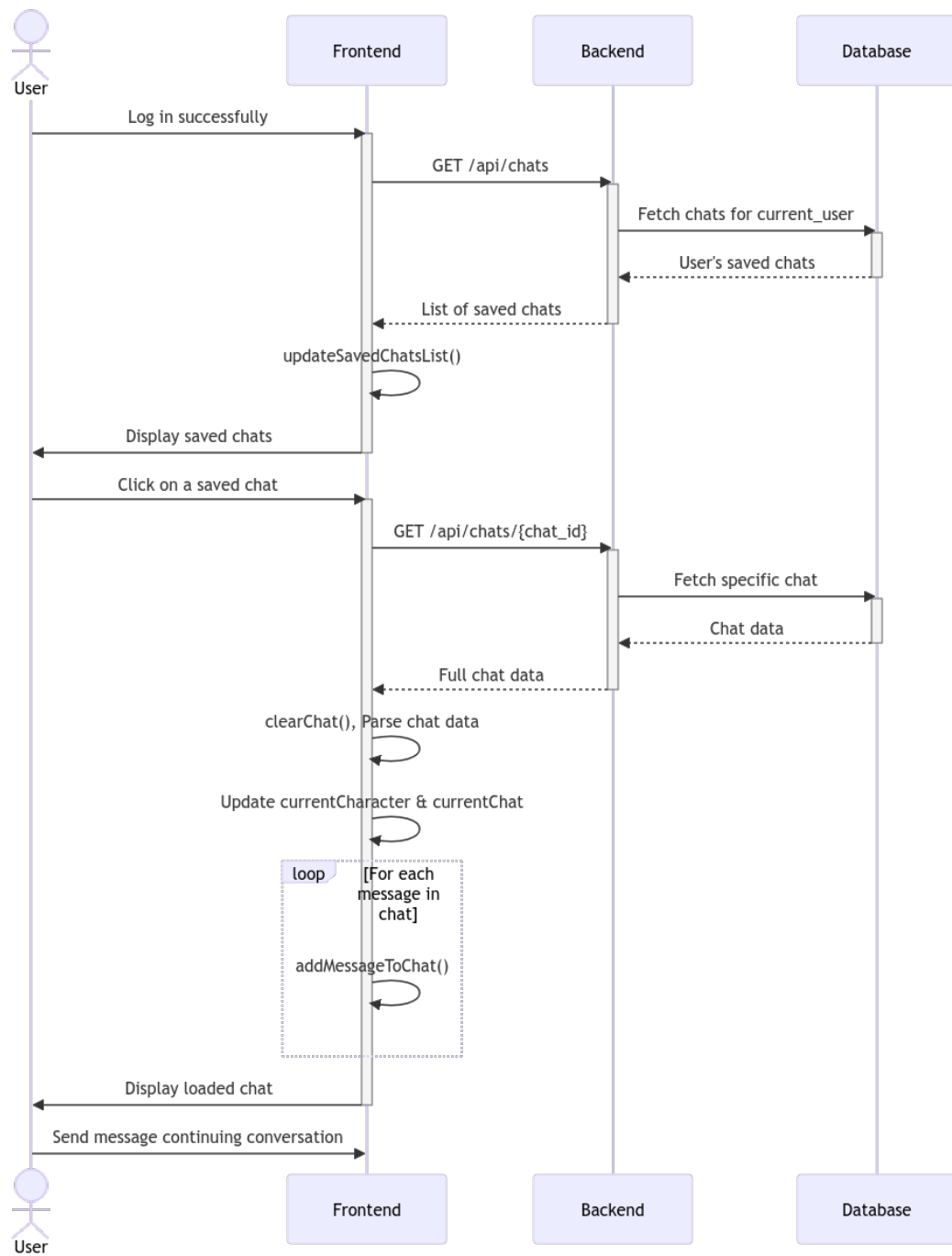


Figure 19: Sequence Diagram: Displaying chat list and loading a chat

4 Evaluation

4.1 Introduction to Evaluation

The evaluation of encompasses a multi-stage approach designed to assess both the fine-tuned language model and the web application interface. The strategy combines quantitative metrics with qualitative user feedback to provide a holistic understanding of the system’s performance (Fine-tuned model and web app), user experience and overall effectiveness.

The evaluation process can be broadly categorized into two main components:

- **Fine-tuned Model Evaluation :** During the model development phase, I employed a set of quantitative metrics including BLEU scores, cosine similarity, LIWC (Linguistic Inquiry and Word Count) analysis and sentimental analysis. These metrics, as detailed in Section (3.2.4.2 Results and Analysis), provided an initial benchmark of the model’s performance in terms of linguistic accuracy, semantic similarity, and characteristic speech patterns. Later, as part of the user study, I conducted A/B testing to compare the fine-tuned Sherlock Holmes model against a baseline non-fine-tuned model. This analysis aimed to assess the model’s authenticity in character portrayal and the improvements in character emulation by fine-tuning with character specific dataset.
- **Web application Evaluation:** The web application underwent rigorous usability testing as part of our user study. This evaluation focused on the application’s ease of use, intuitiveness, and overall user experience

Initially, evaluation was dedicated to the quantitative assessment of the model during its development phase. The remaining efforts were concentrated on the user study, encompassing both the A/B testing of the model and the usability testing of the web application. This user-centric multi-pronged evaluation approach allowed me to not only assess the technical achievements of the project but also to understand the real-world performance of the web application and user perceptions regarding the model.

4.2 User Study

In the following subsections, I will discuss the specifics of the user study , present the results from both the A/B testing and usability evaluations, and provide a detailed discussion of the findings. I will also revisit and expand upon the initial model evaluation results, contextualizing them within the broader framework of the assessments.

4.2.1 Study Design

4.2.1.1 Participants

A total of 7 participants were recruited for this study over a period of 15 days from August 14th to August 28th, 2024. All participants were students at the University of Nottingham, ensuring a relatively homogeneous sample in terms of educational background. Some demographic information was collected, the breakdown was as follows:

- Age distribution:
 - 18-25 years old: 5 participants (71.4%)
 - 25-34 years old: 2 participants (28.6%)
- Familiarity with Sherlock Holmes stories:
 - Very familiar (read multiple stories/watched adaptations): 4 participants (57%)
 - Somewhat familiar (know the basic character): 2 participants (29%)
 - No prior knowledge: 1 participant (14%)

Participant recruitment was conducted through advertisements in university group chats (Discord, WhatsApp) targeting students interested in Computer Science and literature. This method ensured a pool of participants with varying degrees of familiarity with Sherlock Holmes, allowing for a diverse range of perspectives in the evaluation.

4.2.1.2 Materials and Setup

The study utilized two primary components:

1. A/B Testing Setup:

- **Modified Web Application:** A specialized version of the main web application was developed specifically for A/B testing. This version allowed seamless switching between Chatbot A and Chatbot B without revealing their identities to participants.
- **Chatbot Models:**
 - Chatbot A: Non-fine-tuned LLaMA 3.1 8B model
 - Chatbot B: Fine-tuned Sherlock Holmes model based on LLaMA 3.1 8B
- **Interface Design:** Both chatbots were presented with identical user interfaces to prevent bias based on visual cues. The key components of interface included:
 - A chat window displaying the conversation history
 - A text input field for user messages

- A "Send" button to submit messages
- A "Radio" button to alternate between A and B
- A "Context input" section and "Character Switching" drop down menu.
- A "Save" chat button and "Saved Chats" section to go back to particular chats with A or B anytime.
- **User Profiles:**
 - Individual login profiles were created for each participant
 - These profiles allowed for:
 - * Personalized interaction with the system
 - * Secure saving of chat histories with both Chatbot A and Chatbot B
 - * Potential for future evaluation and analysis of individual user interactions
 - Ensured data separation and privacy between participants
- **System Message Configuration:** Both chatbots utilized the same system message structure, including:
 - Initial instruction to act as Sherlock Holmes
 - Description of Sherlock Holmes' characteristics
 - Context for the current scenario
 - Information about the interlocutor (Watson or Mrs. Hudson)
- **Inference Setup:** For both chatbots, each inference request included:
 - The full conversation history (system message + all user and assistant messages)
 - Consistent temperature settings for response generation

2. Web Application for Usability Testing:

- **Full-featured Web Application:** The main web application without any modifications, incorporating all implemented functionalities, was used for the usability testing phase.
- **Key Features:**
 - Chat interface with message history display
 - Model switching between local (fine-tuned) and OpenRouter models
 - Switching between various external models on OpenRouter when source set as Open router (e.g, Gemma, Mistral, Llama)
 - API key setup interface for OpenRouter models
 - Audio playback functionality for chatbot responses

- Custom character creation and selection interface
- Temperature adjustment slider
- Chat saving and retrieval system
- User authentication system (login/logout)

3. Testing Environment:

- **Platform:** Microsoft Teams for screen sharing and remote control
- **Hardware:**
 - My Researcher's PC running the testing setup : RTX4060 8GB GPU, 32GB RAM, i7 13th gen CPU
 - Participants: Various personal devices capable of running Microsoft Teams that interacted remotely with the Reasearcher PC.
- **Network:** Stable internet connection required for both researcher and participants
- **Screen Sharing:** Full HD (1080p) screen sharing to ensure clear visibility of the application interface
- **Remote Control:** Microsoft Teams' remote control feature enabling participants to interact directly with the applications

4. Data Collection Tools:

5. **Questionnaires:** Digital forms created using Microsoft Forms for collecting quantitative and qualitative feedback
6. **Chat Logging:** Separate saving of chat histories for both A/B testing and usability testing phases for each user in the SQLite database with the help of seperate profiles created for each user.

This setup ensured that participants could seamlessly interact with both the A/B testing environment and the full web application, while I could monitor and guide the process effectively. The consistent interface across chatbots in the A/B testing phase minimized potential biases, while the full-featured web application allowed for a thorough evaluation of all implemented functionalities.

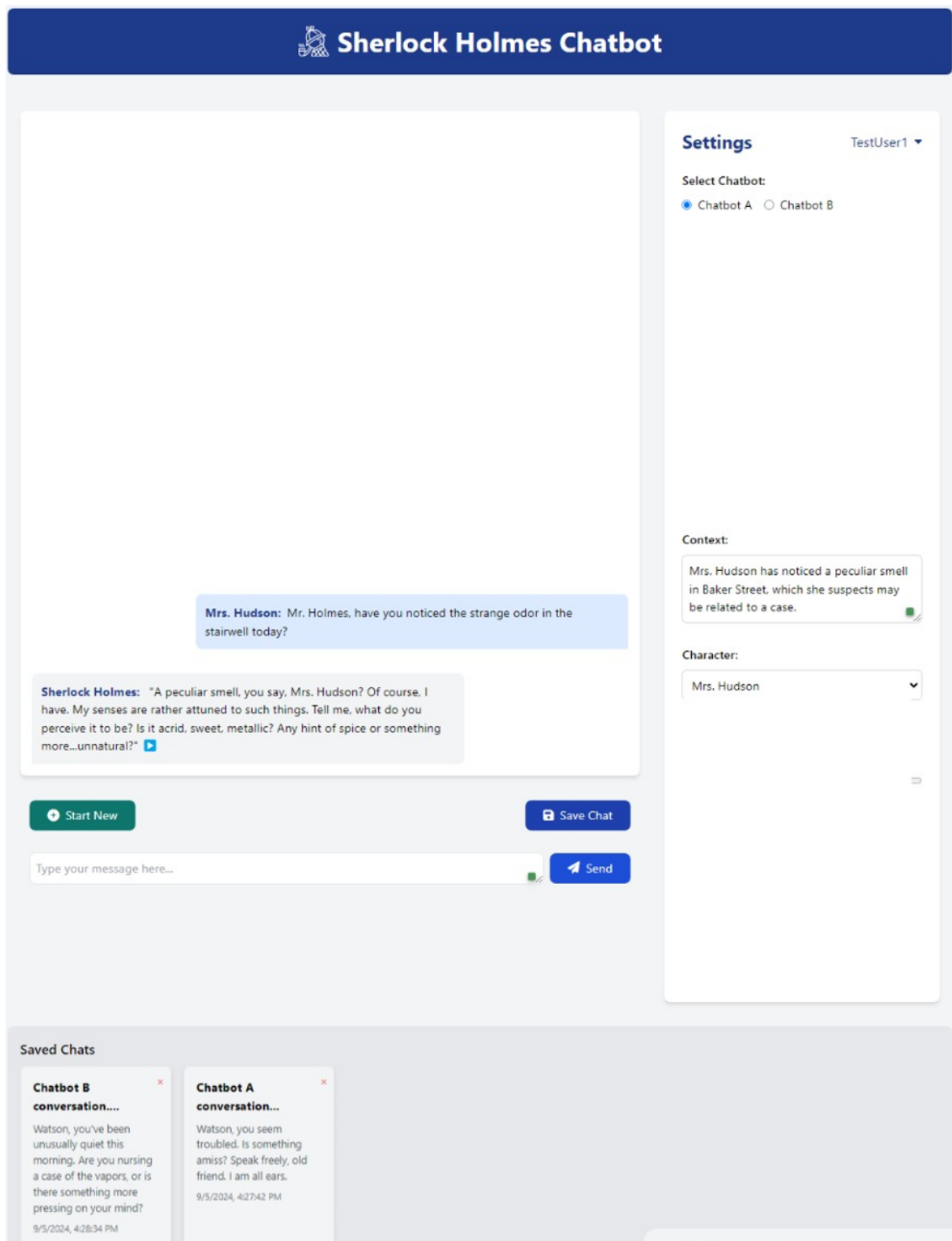


Figure 20: Interface for the A/B testing Modified version of the main web app

4.2.1.3 Procedure

The study was conducted in one-to-one sessions via Microsoft Teams, with an average duration of 45 minutes per session. The researcher screen-shared the application interfaces

while participants were given remote control to interact directly with the systems. The procedure was divided into two main phases:

1. A/B Testing Phase:

- Introduction: Participants were briefed on the study's purpose and structure, emphasizing the importance of their honest feedback.
- Chatbot Assignment): Participants were randomly assigned to start with either Chatbot A or Chatbot B to mitigate order effects. They were not informed which chatbot was the fine-tuned model.
- Scenario Introduction : Two role-playing scenarios were introduced:
 - Scenario 1 (as Dr. Watson): "Holmes, what do you make of the missing ring?". Context : Sherlock Holmes Chatbot A/B Testing.
 - Scenario 2 (as Mrs. Hudson): "Sherlock, a strange man left this package for you earlier; it seemed rather urgent." Context: Mrs. Hudson has noticed a peculiar smell in Baker Street, which she suspects may be related to a case.
- Chatbot Interaction (20 minutes):
 - Participants started with the first chat-bot, role played both scenarios and engaged in 8-9 back-and-forth for both scenarios. After which, the participants switched to the other chat bot and again used both scenarios, engaging in approximately the same number of exchanges.
- Participants were encouraged to maintain character and challenge the chatbot's which would test their authenticity as Sherlock.
- A/B Testing Questionnaire (10 minutes):
 - Participants completed the A/B testing questionnaire.
 - They were encouraged to refer back to the saved conversations for both chat bots or test the chat bots again while filling the questionnaire again if they wanted to test a particular scenario.
 - They were encouraged to ask questions and provided clarification when required.

2. Web Application Usability Testing Phase:

- Introduction to Web Application: After completing the A/B testing questionnaire, participants were introduced to the main web application and its features.
- Guided Exploration:

- Participants were given a brief guided tour of the application’s key features.
- The researcher demonstrated how to access each feature without performing the actions.
- Free Exploration and Testing (6-8 minutes):
 - Participants were encouraged to freely explore and test all features of the web application.
 - Participants were encouraged to think aloud, sharing their thoughts and impressions.
- Usability Questionnaire (8 minutes):
 - Participants completed the web application usability questionnaire.
 - They were allowed to revisit any feature of the application while answering questions.
 - I was available to clarify any questions about the features or the questionnaire.
- Debrief and Additional Feedback (2 minutes):
 - Participants were thanked for their time and asked for any additional comments or feedback.
 - They were informed about the difference between the two chatbots (one being fine-tuned for Sherlock Holmes).

Throughout both phases, the I maintained a neutral stance to avoid influencing participant responses. Participants were consistently encouraged to provide honest feedback and to ask for clarification whenever needed. The structure allowed participants to revisit aspects of the chatbots or web application as required, ensuring that their responses were as informed and accurate as possible.

4.2.1.4 Measures and Metrics

The study employed a set of measures to evaluate both the chatbot models and the web application. These metrics were carefully chosen to assess various aspects of the system’s performance, user experience, and overall effectiveness.

A/B Testing Evaluation Metrics

1. Character Authenticity and Trait Identification

- Overall authenticity rating (5-point Likert scale)
- Selection of convincingly exhibited traits (Multiple-choice and User input)

These metrics assess how well each chatbot embodies the essence of Sherlock Holmes. The authenticity rating provides a general measure, while the trait identification offers insights into specific characteristics that contribute to the portrayal's success.

2. Knowledge and Deductive Reasoning

- Comparative assessment of knowledge demonstration
- Rating of logical deduction ability (5-point Likert scale)
- Frequency of surprising insights (5-point frequency scale)

For evaluating the chatbots' ability to replicate Holmes' knowledge and deductive skills, the metrics aimed to gauge how well they capture these central aspects of his character..

3. Language and Communication Style

- Assessment of Victorian-era language style
- Identification of anachronisms if any
- Rating of iconic phrases and mannerisms usage (5-point Likert scale)
- Effectiveness in incorporating context (5-point frequency scale)

These metrics help in assessing the linguistic authenticity of the chatbots, measuring their ability to maintain period-appropriate language and Holmes' distinctive communication style.

4. Interaction Quality

- Engagement and creativity in responses (Multiple choice and Likert scale)
- Context maintenance rating (5-point Likert scale)
- Adaptation to user's chosen character (5-point Likert scale)

By evaluating ability to maintain engaging, contextually appropriate conversations and adapt to different interlocutors, the aim was to assess the quality of the role-playing experience and the issue with maintaining persona and context over extended conversation highlighted in the Literature Review.

Web Application Usability Evaluation Metrics

1. Navigation and Intuitiveness

- Ease of navigating the chat interface (5-point Likert scale)
- Intuitiveness of switching between models (5-point Likert scale)

These metrics help understand the user-friendliness of the application's core functionalities, which is crucial for user adoption and satisfaction.

2. Feature-Specific Usability

- User-friendliness of API key setup (5-point Likert scale)
- Enhancement from audio playback (5-point Likert scale)
- Ease of creating and using custom characters (5-point Likert scale)
- Clarity of temperature adjustment feature (5-point Likert scale)
- Satisfaction with conversation saving/retrieval (5-point Likert scale)

By evaluating the usability of specific features, the metrics aimed to identify areas of strength and potential improvement in the application's design.

3. Overall Application Assessment

- Identification of most useful features (Ranking)
- Presence of technical issues (Yes/No with description)
- Overall design and aesthetic rating (5-point scale)

These metrics give us an overarching assessment of the application, highlighting its most valuable aspects and any technical challenges faced by users.

Overall User Experience

- Overall preference between chatbots (Multiple choice)
- Enjoyment rating (10-point scale)
- Open-ended feedback on impressive aspects and suggested improvements

These metrics provide an idea of the user experience, capturing both quantitative preferences and qualitative insights for potential future improvements. **Demographic Information**

- Age range
- Familiarity with Sherlock Holmes stories

This information allows for analysis of how user characteristics might influence their experience with the system, potentially revealing insights for targeted improvements or marketing strategies.

These metrics were designed to provide a full evaluation of both the AI models' performance in character emulation and the usability of the web application interface.

4.2.1.5 Limitations

The primary limitation of this study was the relatively small sample size of 7 participants, which was due to time constraints in the recruitment process. While this sample size allows for valuable insights, it may limit the generalizability of the results. Additionally, the participant pool being exclusively university students may not represent the full diversity of potential users for such a system.

4.2.2 Results

This section presents the findings from both the A/B testing of the chatbot models and the usability evaluation of the web application. The results are organized into quantitative data analysis and qualitative feedback summary.

4.2.2.1 A/B testing Results

Character Authenticity and Trait Identification

Overall Authenticity Ratings:

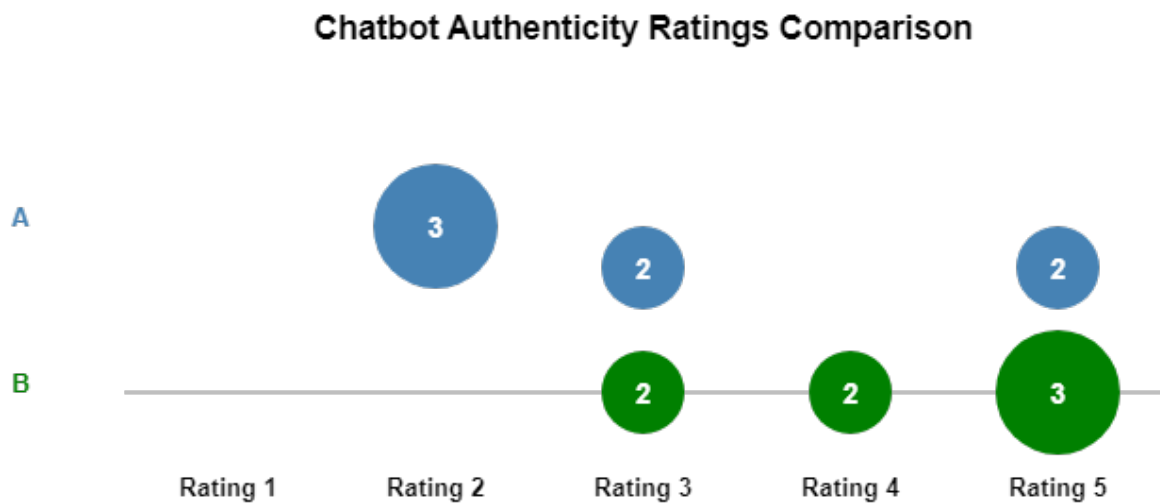


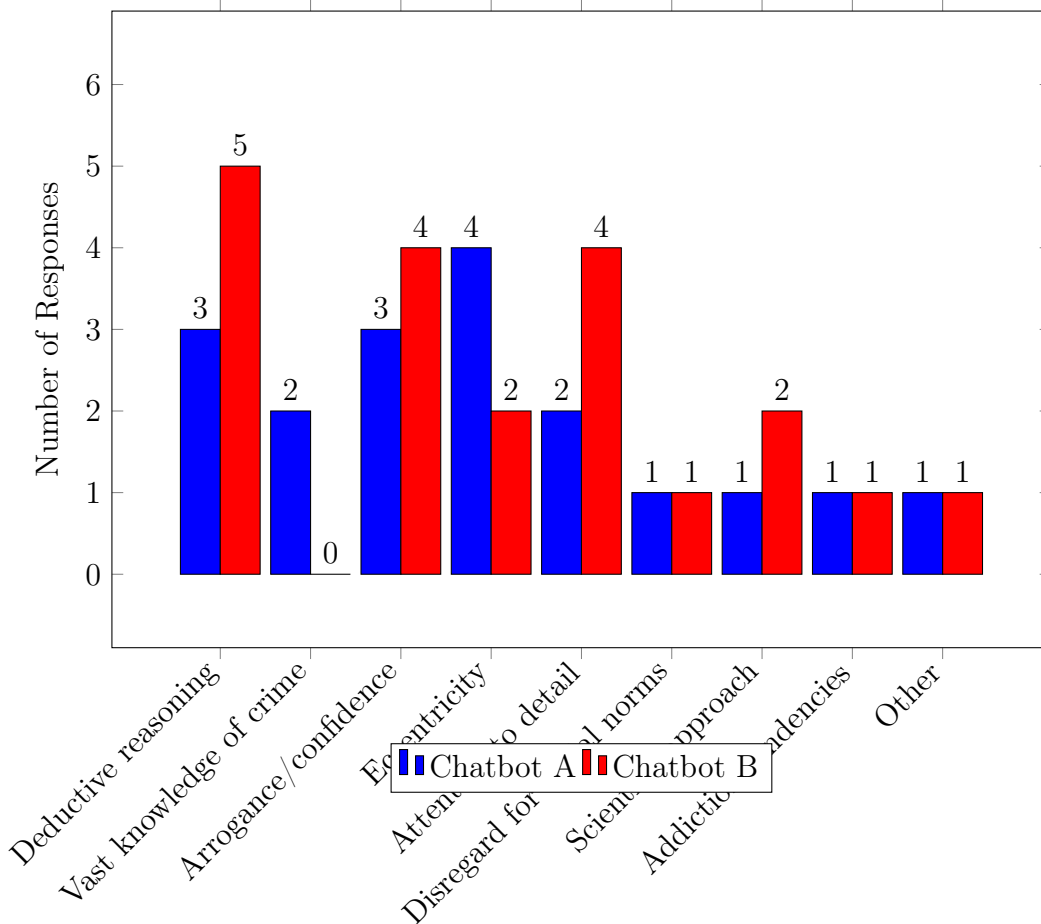
Figure 21: Chat-bots Authenticity Dot plot. The numbers within the circles represent the number of participants who gave the respective rating

	Chatbot A	Chatbot B
Mean Authenticity Rating	3.29	4.14
Standard Deviation	1.25	0.84

Table 3: Mean Authenticity Ratings for Chatbots

Chatbot B, the fine-tuned model, received a higher mean authenticity rating (4.14) compared to Chatbot A (3.29), suggesting that the fine-tuning process improved the perceived authenticity of the Sherlock Holmes character portrayal.

Top Traits Exhibited:



The top traits exhibited by each chatbot, as identified by participants, were:

- Chatbot A: Arrogance/confidence, Eccentricity, Attention to detail
- Chatbot B: Deductive reasoning, Arrogance/confidence, Attention to detail

Notably, Chatbot B was more consistently associated with deductive reasoning, a key characteristic of Sherlock Holmes.

Role-playing and Character Adaptation

How well did each Sherlock persona in chatbot adapt its responses to your chosen character that the user was roleplaying as?

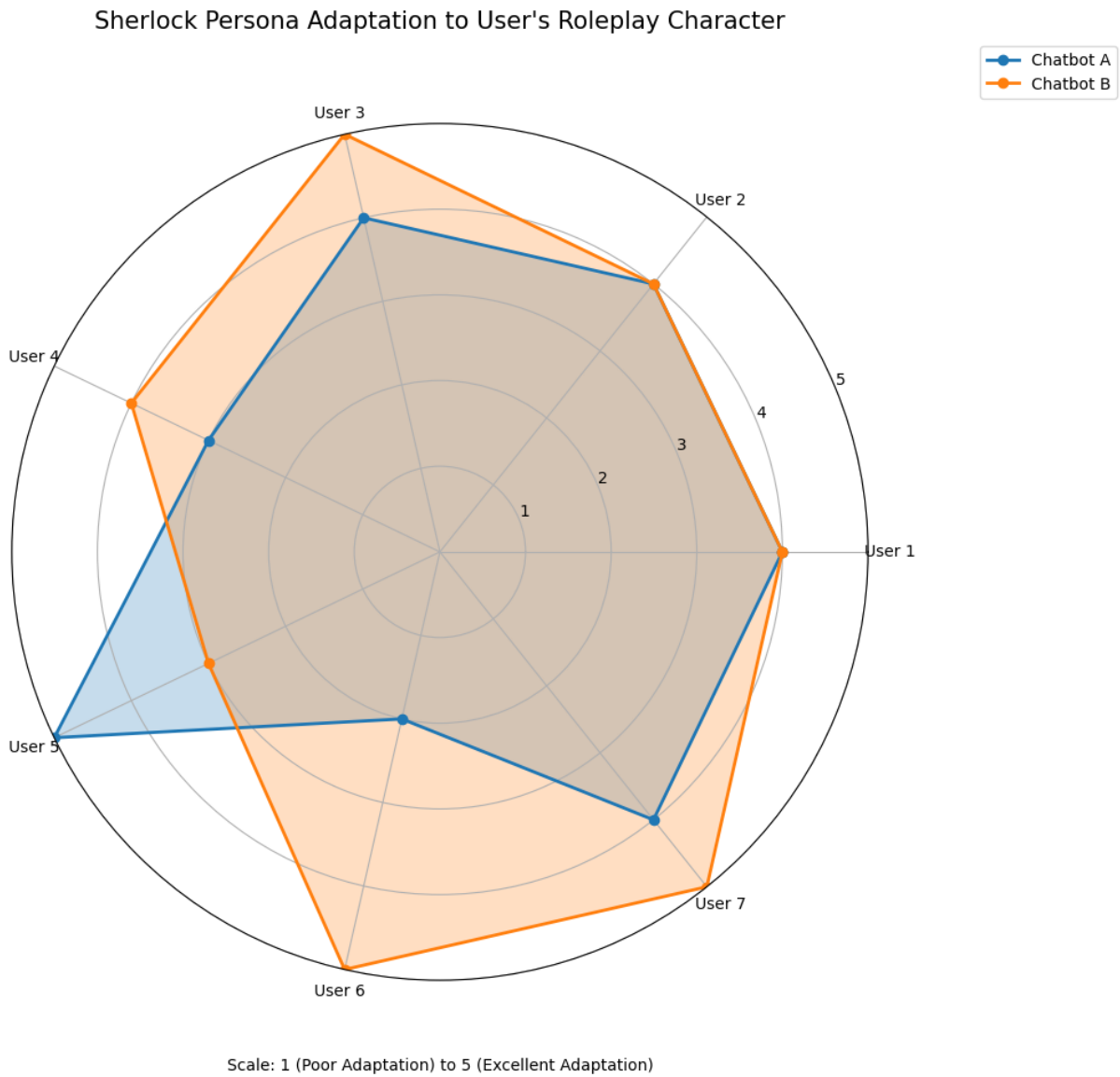


Figure 22: Radar Chart: Sherlock Persona Adaptation

Adaptation to the chosen character was rated on a scale of 1 to 5:

	Chatbot A	Chatbot B
Mean Adaptation Rating	3.71	4.29
Standard Deviation	0.88	0.76

Table 4: Mean Ratings for Adaptation to Chosen Character

Chatbot B showed a slightly superior adaptation to user-chosen characters, indicating improved contextual understanding and responsiveness to different interlocutors

Knowledge and Deductive Reasoning

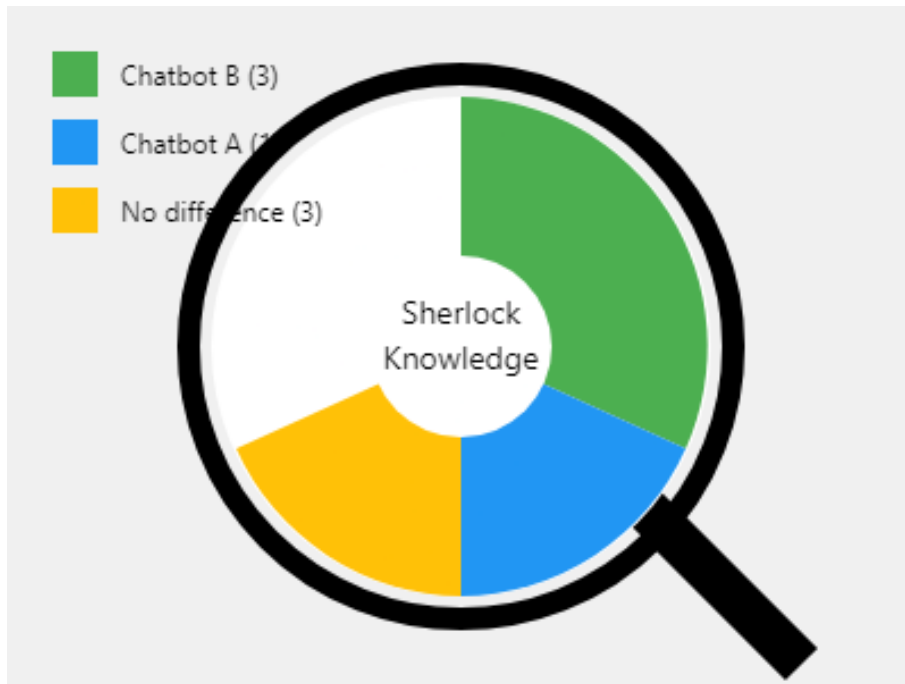


Figure 23: Sherlock Knowledge Comparison

When asked which chatbot demonstrated better *knowledge of Sherlock Holmes stories and characters*:

- 42.86% (3) chose Chatbot B
- 42.86% (3) noticed no significant difference
- 14.28% (1) chose Chatbot A
-

The ability to *make logical deductions* was rated on a scale of 1 to 5:

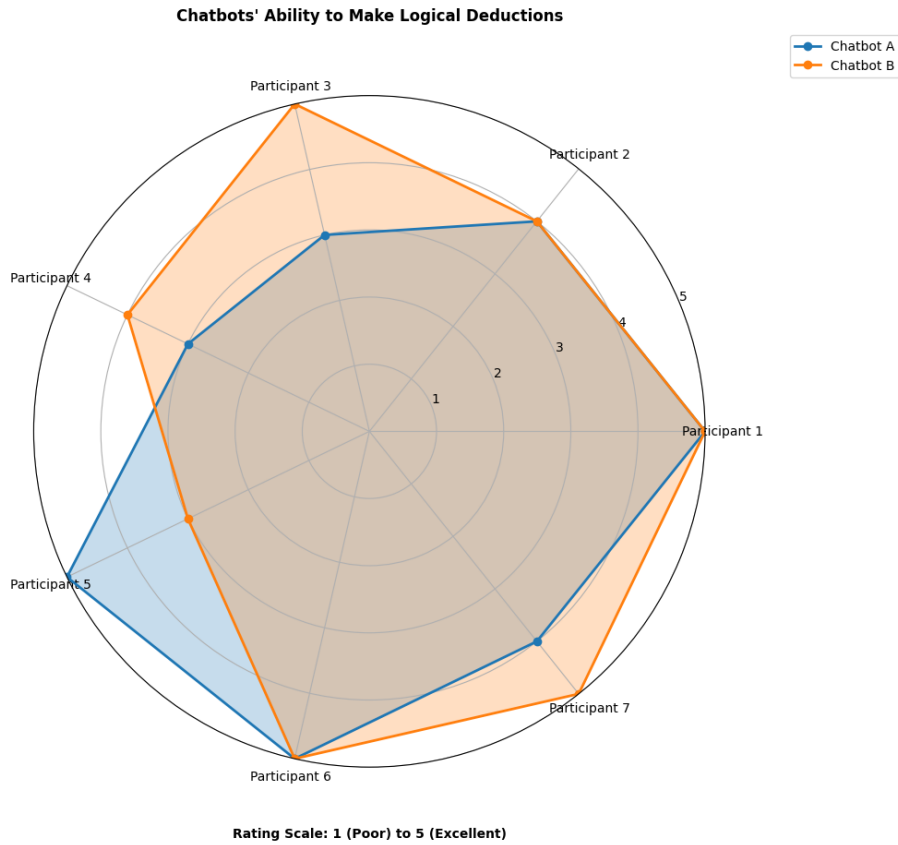


Figure 24: Enter Caption

	Chatbot A	Chatbot B
Mean Deduction Rating	4.14	4.43
Standard Deviation	0.90	0.79

Table 5: Mean Logical Deduction Ratings for Chatbots

While both chatbots demonstrated strong deductive abilities, Chatbot B was more often recognized for its knowledge of Sherlock Holmes stories, suggesting better integration of character-specific information

Mystery Solving Satisfaction

All participants presented mysteries or cases for the chatbots to solve. Satisfaction with each chatbot's approach was rated on a scale of 1 to 5:

	Chatbot A	Chatbot B
Mean Satisfaction Rating	3.43	4.14
Standard Deviation	1.05	0.90

Table 6: Mean Satisfaction Ratings for Mystery Solving

Users were notably more satisfied with Chatbot B's approach to solving mysteries, indicating a more authentic problem-solving style reminiscent of Sherlock Holmes

Surprising Insights

Frequency of surprising insights was rated on a scale from "Rarely" to "Very Often":

Frequency	Chatbot A	Chatbot B
Rarely	0	0
Sometimes	4	3
Often	2	3
Very Often	1	1

Table 7: Frequency of Surprising Insights

Both chatbots provided surprising insights at similar frequencies.

Provided context Incorporation

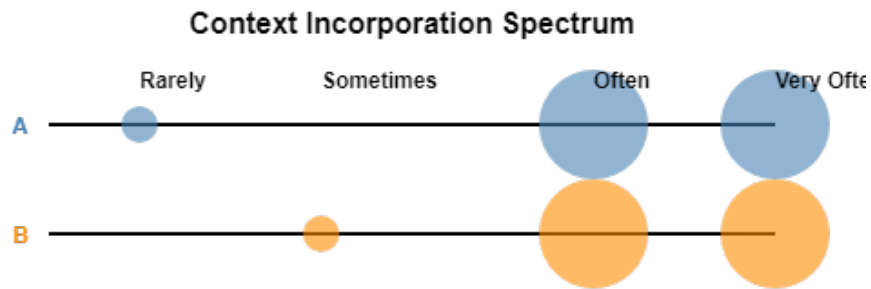


Figure 25: Context Incorporation Comparison

Effectiveness in incorporating provided context was rated from "Rarely" to "Very Often":

Frequency	Chatbot A	Chatbot B
Rarely	1	0
Sometimes	0	1
Often	3	3
Very Often	3	3

Table 8: Frequency of Effective Context Incorporation

Language Style and Anachronisms

Regarding *Victorian-era language style*:

- 42.86% (3) found Chatbot B more reminiscent of the Victorian era
- 42.86% (3) found Chatbot A more reminiscent of the Victorian era
- 14.28% (1) noticed no significant difference

Anachronisms or *out-of-place modern language*:

- 85.71% (6) noticed no anachronisms in any chatbot.

- 14.29% (1) noticed anachronisms in Chatbot A

Use of Iconic Phrases and Mannerism

Rated on a scale of 1 to 5:

	Chatbot A	Chatbot B
Mean Rating	3.71	4.29
Standard Deviation	1.03	0.76

Table 9: Mean Ratings for Use of Iconic Phrases and Mannerisms

Engagement and Creativity

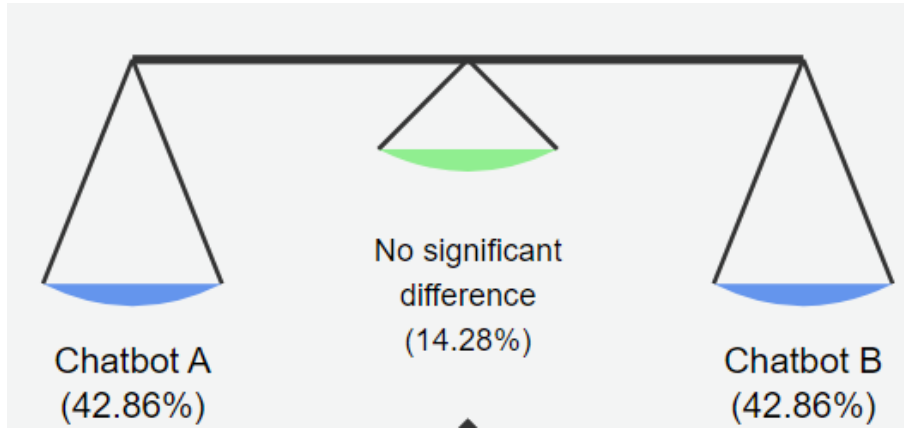


Figure 26: Which Response is more Creative?

When asked which version provided more engaging and creative responses:

- 42.86% (3) chose Chatbot B
- 42.86% (3) chose Chatbot A
- 14.28% (1) found no significant difference

When asked if either chatbot surprised them with its responses in a positive way:

- 28.57 (2) were surprised by both chatbots
- 42.86% (3) were surprised by Chatbot B
- 14.28% (1) was surprised by Chatbot A
- 14.28% (1) were not surprised by either chatbot

Context Maintenance

Ability to maintain context throughout the conversation, rated on a scale of 1 to 5:

	Chatbot A	Chatbot B
Mean Rating	4.14	4.43
Standard Deviation	0.83	0.72

Table 10: Mean Ratings for Context Maintenance

Overall Preference

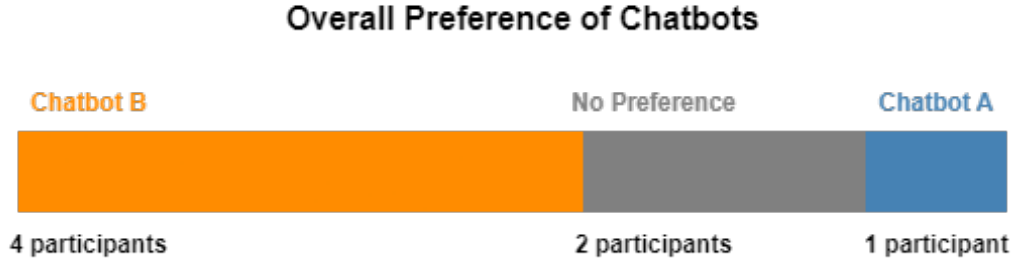


Figure 27: Overall Preference of Chatbots

- 71.43% (5) preferred Chatbot B overall
- 28.57% (2) had no preference

The majority preference for Chatbot B aligns with its higher scores across other metrics.

Identification of Fine-tuned Model

When asked which version they thought was the fine-tuned model:

- 85.71% (6) correctly identified Chatbot B as the fine-tuned model
- 14.29% (1) incorrectly identified Chatbot A as the fine-tuned model

Reasons for identifying the fine-tuned model included:

- More consistent personality (3)
- More authentic Victorian-era language (1)
- More detailed knowledge of Sherlock Holmes stories (1)
- Better deductive reasoning (1)

The high accuracy in identifying Chatbot B as the fine-tuned model, primarily due to consistent personality, suggests that the fine-tuning process created a noticeably more authentic Sherlock Holmes persona **Overall Enjoyment**

Enjoyment was rated on a scale of 1 to 10:

	Chatbot A	Chatbot B
Mean Enjoyment Rating	8.14	9.00
Standard Deviation	1.68	1.15

Table 11: Mean Enjoyment Ratings

While both chatbots were highly enjoyable, the consistently higher ratings for Chatbot B indicate that the fine-tuned model enhanced the overall user experience

Key-observations from Open-ended Feedback

Chatbot A (Non fine-tuned model): Some users found it tried too hard to interpret every scenario as a mystery. Occasionally used unnatural expressions or actions (e.g., "smirking", "leaning against the mantle"). Generated longer, more elaborate responses, which some users appreciated for storytelling but expressed it felt unnatural in the context of having conversation.

Chatbot B (Fine-tuned model): Consistently noted for better adherence to context and more natural conversational flow. Praised for its ability to adapt to different interlocutors (e.g., Watson vs. Mrs. Hudson). Some users found its responses too concise and wished for more elaboration especially when roleplaying as Mrs. Hudson. Perceived as more authentic in capturing Sherlock's mannerisms and speech patterns.

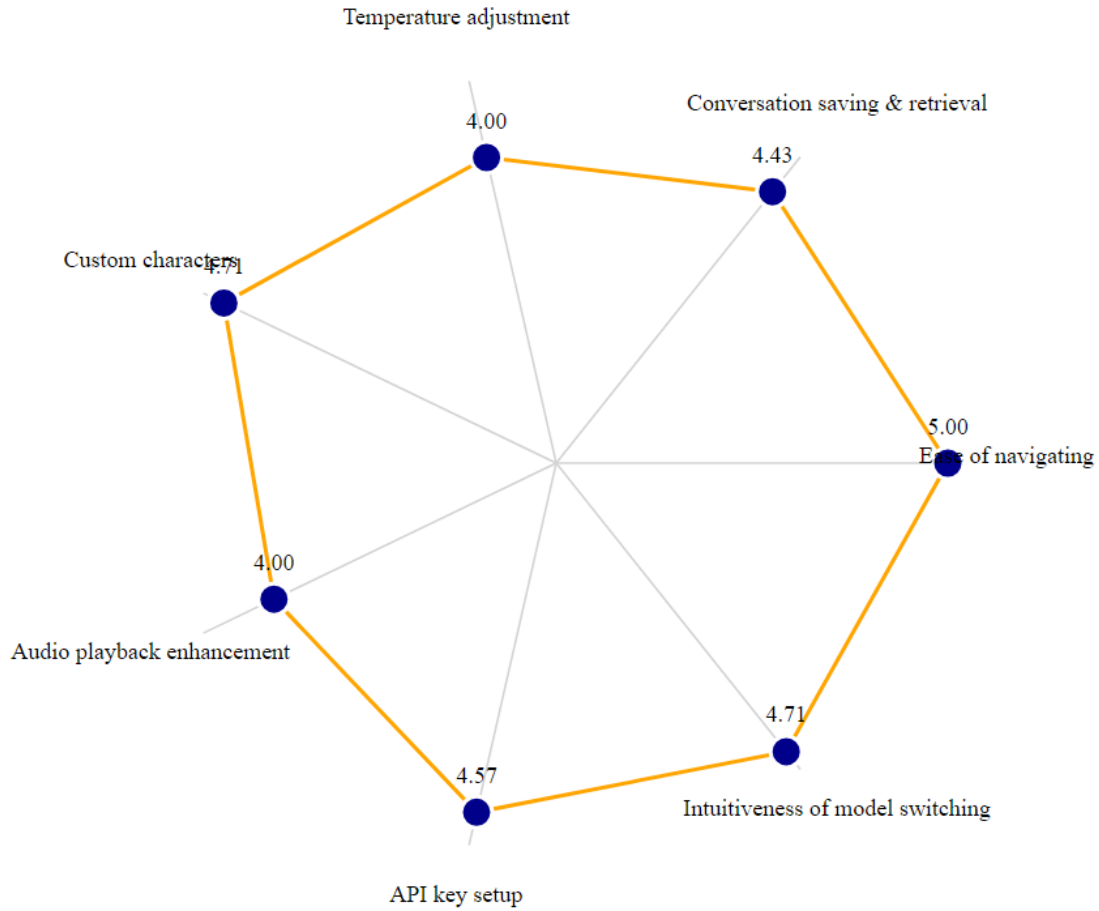
Both chatbots were praised for their deductive reasoning and attention to detail. Users enjoyed the role-playing aspect and found it hard to stop interacting with the system.

4.2.2.2 Website Usability testing Results

Navigation and Intuitiveness

- Ease of navigating the chat interface: Mean rating of 5.00 (SD = 0.00) on a 5-point scale
- Intuitiveness of switching between models: Mean rating of 4.71 (SD = 0.49) on a 5-point scale

The perfect score for chat interface navigation and near-perfect score for model switching indicate a user-friendly design. The consistency across users (as shown by low standard deviations) suggests that the interface is intuitive for users of somewhat varying technical backgrounds (Literature and CS students) .



Chatbot Features Ratings Constellation

Figure 28: Webapp features and usability rating constellation

Feature-Specific Usability

Feature	Mean Rating	SD
User-friendliness of API key setup	4.57	0.53
Experience enhancement from audio playback	4.0	1.07
Ease of creating custom characters	4.71	0.49
Clarity of temperature adjustment	4.00	1.41
Satisfaction with conversation saving/retrieval	4.43	0.79

Table 12: Mean Ratings for Feature-Specific Usability (5-point scale)

All features received positive ratings, with custom character creation and API key setup being particularly well-received. The temperature changing feature, while still positively

rated, had the lowest mean score and highest variability, suggesting room for improvement or differing user preferences.

Most Useful features

Participants were asked to rank the features of the web application from most to least useful. The following table shows how often each feature was ranked in the top 5 most useful features by participants:

Feature	Times Ranked in Top 5
Chat interface	7
Custom characters	7
Model switching	6
Conversation saving/retrieval	6
Login/logout system	4
Audio playback	3
API key setup	2
Temperature adjustment	1

Table 13: Feature Usefulness Rankings (Top 5)

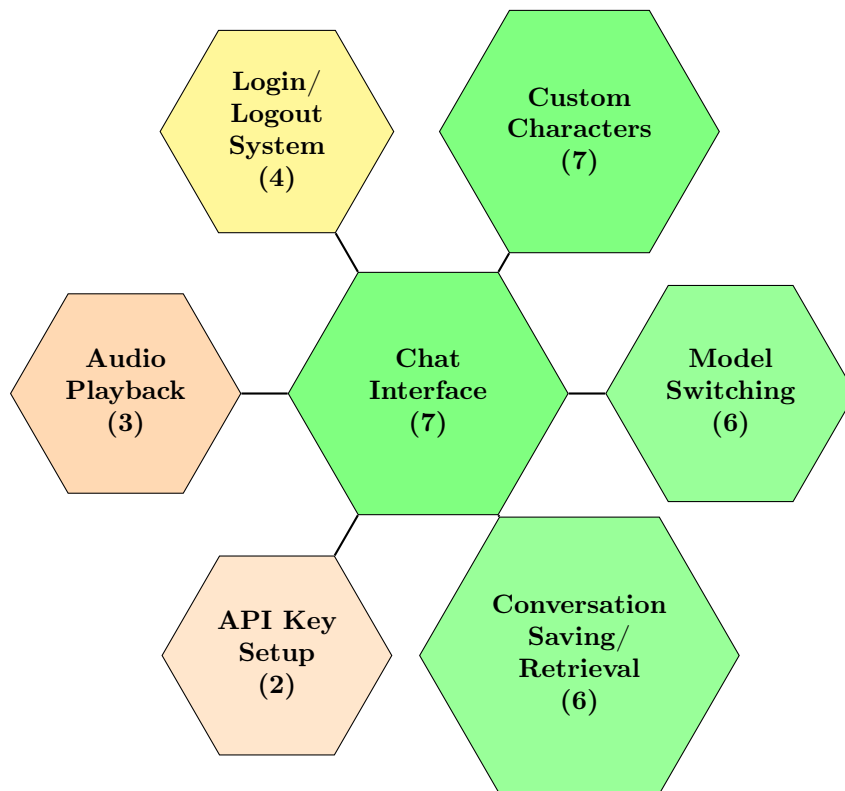


Figure 29: Honeycomb Visualization of Most Liked Features (Size and Color Indicate Popularity)

Key-observations from Open-ended Feedback for Usability Some users found the purpose of the temperature adjustment feature unclear and suggested better explanation or documentation. While generally well-received, the audio playback feature had mixed

reviews, with some users finding it to cause friction in their experience as the wait time for audio to generate for long dialogues was upwards of 5 seconds. A few users suggested implementing an auto-save feature for chats to improve the user experience as the only way to save is to use press the save button manually every-time a chat needs to be saved. A few users suggested implementing suggested questions or prompts to guide the conversation. The chat interface was universally praised for its intuitiveness and ease of use. Model switching functionality was appreciated for its simplicity and effectiveness. Conversation saving and retrieval system was noted as a useful feature for maintaining continuity.

4.3 Fine-tuned model Evaluation and Discussion

The following discussion synthesizes insights from the initial quantitative evaluation and the subsequent A/B testing to provide a comprehensive analysis of the model’s performance.

4.3.0.1 Quantitative Metrics vs. User Perceptions

The initial quantitative evaluation using BLEU scores, cosine similarity, and LIWC analysis provided early insights into the model’s capabilities. The moderate BLEU score (0.52 ± 0.34) on the training set suggested that the model learned to reproduce Holmes’ phrasings while introducing variations, avoiding mere memorization. However, the significant drop in BLEU scores for the unseen test set (0.126 ± 0.022) initially raised concerns about generalization. Interestingly, this apparent limitation in generalization based on traditional NLP metrics contrasted sharply with user perceptions in the A/B test, where the fine-tuned model (Chatbot B) was overwhelmingly preferred (71.43%). This discrepancy highlights an important point: traditional NLP metrics may not fully capture the essence of successful character emulation. The fine-tuned model’s ability to maintain key linguistic patterns (as shown by consistent LIWC and sentiment analysis scores across seen and unseen data) appears to be more important for perceived authenticity than exact phrase matching. In the user study, the fine-tuned model receiving high ratings for logical deduction (4.43 out of 5) and mystery-solving satisfaction (4.14 out of 5) even in novel scenarios presented during A/B testing. This suggests that while traditional NLP metrics show a generalization gap, the model still captures enough of the character’s essence to perform well in user interactions.

4.3.0.2 Authenticity and Character traits

The A/B testing results strongly validated the effectiveness of the fine-tuning process in capturing Holmes’ essence. The fine-tuned model consistently outperformed the base model in perceived authenticity (4.14 ± 0.84 vs. 3.29 ± 1.25) and was more frequently

associated with key Holmesian traits, particularly deductive reasoning and attention to detail. This shows fine-tuning emphasized crucial character attributes, creating a more convincing portrayal of Sherlock Holmes. The improvement in replicating Holmes’ distinctive speech patterns and mannerisms, as evidenced by higher ratings for the use of iconic phrases (4.29 ± 0.76 vs. 3.71 ± 1.03), further reinforces the success of the fine-tuning process in better capturing of the character’s unique linguistic style.

4.3.0.3 Context Adaptation and Persona

Consistency One of the most significant achievements of the fine-tuned model was its superior performance in context adaptation and persona maintenance. The model demonstrated a greater ability to adjust its responses to different interlocutors and maintain conversational context, as evidenced by higher ratings in character adaptation (4.29 vs 3.71) and context maintenance (4.43 vs 4.14). This addresses a key challenge identified in the literature review. The improvement suggests that character-specific fine-tuning can enhance a model’s ability to maintain consistent persona across extended conversations, with broader implications for developing more coherent and immersive AI characters for various applications.

4.3.0.4 Creativity vs. Authenticity: A Delicate Balance

An interesting finding from the A/B testing was the apparent trade-off between authenticity and creativity. While the fine-tuned model (Chatbot B) was perceived as more authentic, with 71.43% of participants preferring it overall, it didn’t significantly outperform the base model in terms of creativity. When asked which version provided more engaging and creative responses, participants were evenly split: 42.86% chose Chatbot B, 42.86% chose Chatbot A, and 14.28% found no significant difference. Similarly, the frequency of surprising insights was comparable between the two models, with both receiving mostly ‘Sometimes’ to ‘Often’ ratings. This raises a fundamental question in character emulation: How can we improve authenticity without constraining the model’s creative potential? This paradox might stem from the nature of fine-tuning itself. By focusing the model on a specific character’s patterns, we may inadvertently limit its ability to generate novel or unexpected responses. Finding the right balance between staying true to character and maintaining engaging unpredictability is a key challenge for future research in this field.

4.3.0.5 Challenges and limitations

Despite the overall success of the fine-tuned model, several challenges and limitations were identified, some of which are already detailed above:

Generalization to Novel Scenarios: The significant drop in BLEU and cosine similarity

scores for unseen data highlights a potential issue. While the model excels with familiar contexts, it may struggle with entirely new scenarios in terms of accurately representing how the character would behave in new or untested situations.

Some users mentioned that the fine-tuned model’s responses were occasionally more concise than those of the non-fine-tuned model, particularly in role-playing as Mrs. Hudson. This suggests a need for improved calibration in response length, possibly due to the fact that the dataset it was trained on featured shorter dialogues with that specific character, which it learned to replicate.

The even split in perceived creativity between the fine-tuned and base models indicates that while authenticity improved, creative potential may have been constrained. The similar frequencies of surprising insights between both chatbots suggest that the fine-tuning process, while improving overall authenticity, did not significantly enhance the model’s ability to generate unexpected or particularly insightful responses.

/

4.3.0.6 Ethical Challenges and Broader Implication of Character Emulation

The high enjoyment ratings and strong preference for the fine-tuned model raise important considerations about user expectations and the ethical implications of highly convincing character emulations. As these models become more authentic and engaging, users may develop stronger emotional connections to them. This could lead to concerns about emotional manipulation or the blurring of lines between AI and human interaction.

The emulation of Sherlock Holmes, a public domain character, potentially benefited from pre-existing knowledge in the base model, as it was likely trained on Holmes-related data among others even before fine-tuning. This raises two challenges: a) *Difficulty in Assessing True Fine-tuning Impact*: The pre-existing knowledge of Sherlock Holmes in the base model makes it challenging to isolate and quantify the exact improvements brought about by the fine-tuning process. This complicates the evaluation of the fine-tuning technique’s effectiveness for entirely new characters. b) *Ethical and Privacy Concerns for Emulating Real Individuals*: Replicating the success of this project with a completely new or contemporary person poses significant ethical challenges. Issues of privacy, consent, and potential misuse become paramount when considering the emulation of real individuals, those who are living or deceased. This challenge extends to:

- Obtaining proper consent for using personal data
- Ensuring accurate representation without mischaracterization
- Preventing potential abuse or unauthorized use of a person’s likeness
- Navigating legal restrictions on personal data use
- Addressing concerns about the posthumous use of an individual’s personality

4.3.0.7 Future Directions

Based on the above evaluation, several avenues for future improvement and research emerge:

- **Expanded Training Data:** Incorporating a wider range of Sherlock Holmes stories and adaptations could lead to improving the model’s ability to generalize to new scenarios.
- **Dynamic Response Calibration:** Developing mechanisms to adjust response length and more details based on the interlocutor and conversational context could enhance the model’s versatility.
- **Era-Specific Language Models:** Pre-training on Victorian-era literature before fine-tuning could improve the consistency of period-appropriate language.
- **Exploring techniques to boost the model’s ability to generate surprising insights while maintaining character authenticity.** This could be improved by incorporating a large parameter sized models like 70 billion or 408 billion parameters sized versions Llama 3.1 in the pipeline.
- **Multi-Character Interaction:** Extending the dataset so the model can handle interactions between multiple characters from the Sherlock Holmes universe at the same time could create more engaging scenarios.

4.4 WebApp usability Evaluation and Discussion

The usability testing of the Sherlock Holmes AI Chatbot web application revealed overall positive user experiences, with high ratings across various features. The evaluation encompassed navigation, specific feature usability, and overall application assessment. Overall, participants embraced the concept enthusiastically, their experiences highlighting both the strengths of my approach and areas where I might need to reconsider my design choices.

4.4.0.1 Navigation and Core Functionality

One of the most striking outcomes was the unanimous praise for the chat interface’s navigability. Achieving a perfect score (5.00 ± 0.00) in ease of use is rare in usability testing, and it highlights the intuitive design I implemented. This success likely stems from my decision to mimic familiar messaging app layouts, providing users with a familiar environment for their interactions with Holmes. The high ratings for model switching functionality (4.71 ± 0.49) were particularly gratifying, given the technical complexity behind this feature. It seems that the efforts to abstract the complexity of inferencing and

switching into a user-friendly interface paid off, allowing even those without a deep technical background such as the literature students to experiment with different AI models easily.

4.4.0.2 Feature-Specific Usability

The custom character creation feature received good reception (4.71 ± 0.49), suggesting a strong desire for being able to role-play as any character, including the user themselves. The high score suggests that the interface for creating and managing custom characters was both intuitive and engaging. The positive response to Conversation Saving/Retrieval feature highlights its importance in maintaining continuity across user sessions. However, the slightly lower score and higher variability suggest there might be room for improvement, possibly in the form of an auto-save feature as suggested by some users. While generally well-received, the audio playback feature showed high variability (4.0 ± 1.07) in user satisfaction. The concept of hearing Holmes speak added immersion, but performance issues (in terms of waiting time for the audio to generate), particularly for longer responses, created friction in the user experience. The temperature adjustment feature received the lowest mean score and highest variability. The mixed reception suggests that while some users appreciated the fine-grained control, others found it confusing or unnecessary.

The chat interface and custom character creation were unanimously ranked in the top 5 most useful features by all participants. Model switching and conversation saving/retrieval were also highly valued, appearing in the top 5 for 6 out of 7 participants. This aligns with the core functionality of the application and validates my focus on these key features during development.

4.4.0.3 Challenges and Limitations

- The varied reception of advanced features like temperature adjustment highlights the challenge of catering to users with different levels of technical expertise.
- Performance Bottlenecks: The audio playback feature, while appreciated, revealed performance issues that disrupted the flow of conversation, particularly for longer responses.
- The suggestion for an auto-save feature indicates that my current manual saving process might be cumbersome for some users, potentially leading to lost conversations.
- Guiding User Interactions: Some users suggested implementing prompts or suggested questions, indicating a need for more guidance in navigating conversations, especially for those less familiar with the Sherlock Holmes universe.

- Feature Discoverability: While core features were well-received, the mixed response to some advanced features suggests that I might need to improve their visibility or explanation within the interface.
- As a solo project without funding, I was unable to create a live, publicly accessible website. This limitation was particularly significant given the resource-intensive nature of hosting inference capabilities for fine-tuned models. A live website would have allowed for broader user testing and real-world application of the chatbot, potentially yielding more diverse feedback and use cases. The inability to deploy live also limited the potential for a bigger turnout for the user study and long-term usage analysis.
- Due to resource limitations, I had to work with lower parameter-sized language models. While these models performed well, larger models would likely provide better results.

The limitations highlight the gap between what's theoretically possible with cutting-edge AI technologies and what's practically achievable with limited resources. Despite these limitations, the project still managed to provide valuable insights into character emulation and user interaction with AI chatbots.

5 Conclusion and Reflection

What began as an ambitious exploration into character emulation using large language models evolved into a specific focus on creating an authentic Sherlock Holmes chatbot while exploring the ethical implications and technical challenges of such systems.

The primary objective was to develop a fine-tuned LLM capable of authentically emulating Sherlock Holmes. This goal was successfully met, as evidenced by the A/B testing results. The fine-tuned model achieved a mean authenticity rating of 4.14 compared to 3.29 for the non-fine-tuned model. Participants consistently identified key "Holmesian" traits in the fine-tuned model, particularly deductive reasoning and attention to detail. The model's ability to adapt its responses to different interlocutors (e.g., Watson vs. Mrs. Hudson) further demonstrated its success in capturing the essence of Holmes' character.

I successfully created a fine-tuning pipeline operable on limited computational resources, addressing the challenge of accessibility for researchers and developers with constrained resources. By leveraging Parameter-Efficient Fine-Tuning (PEFT) techniques, specifically Low-Rank Adaptation (LoRA), I was able to fine-tune a 4-bit quantized version of the LLaMA 3.1 8B model. This approach significantly reduced computational requirements while maintaining high performance, showing that it is a viable path for character emulation with limited resources.

The development of the user-friendly web interface for interacting with the Sherlock Holmes AI was another aspect of the project that I found particularly rewarding. The perfect score (5.00 ± 0.00) for chat interface navigation and near-perfect score (4.71 ± 0.49) for model switching functionality highlighted the success in creating an intuitive and engaging platform. Features like custom character creation, conversation saving/retrieval, and audio playback, which I implemented to enhance the user experience, were well-received and added depth to the interaction with the AI.

My evaluation approach, combining traditional NLP metrics with comprehensive user studies, provided me with valuable insights that I hadn't anticipated at the project's outset. The discrepancy I observed between quantitative metrics (such as BLEU scores) and user perceptions was particularly intriguing. It made me realise the complexity of evaluating character emulation and reinforced the importance of human feedback in assessing the quality of AI-generated content. By choosing Sherlock Holmes, a public domain character, I navigated the ethical and legal challenges associated with using copyrighted or personal data (as explored in Literature Review). This approach allowed me to focus on technical challenges while demonstrating a methodology that can be applied to a wide range of characters. However, the project also revealed important challenges and limitations that I hadn't fully anticipated. The trade-off between authenticity and creativity, the need for improved generalization to novel scenarios, and the potential ethical implications of highly convincing character emulations all present avenues for future research and development. As a solo developer, I also faced limitations in terms of deploying a live, publicly accessible website and working with larger, more powerful language models. These constraints, while challenging, provided me some valuable lessons in working efficiently within limited resources.

In conclusion, the project has not only met its initial objectives but also opened up new questions and possibilities that I'm eager to explore further. It has demonstrated to me the potential of combining advanced NLP techniques with thoughtful interface design to create engaging and fun experiences for other people. The insights I've gained from this project will undoubtedly influence my future work especially related to AI, LLMs and Human-AI interactions. This project has also been a significant milestone in my academic journey as I'm struck by the journey it has taken me on over the past three months. It has not only improved my technical skills but also deepened my understanding of the broader implications of AI in society. . The ethical considerations highlighted by this project, particularly regarding the emulation of real individuals and the potential for emotional manipulation, have deeply impressed upon me the need for continued careful consideration as these technologies advance. As I continue, I'm more committed than ever to developing these technologies in a way that is both innovative and responsible.

References

- (2024). Openrouter: Unified interface for large language models. <https://openrouter.ai>. Accessed: 2024-09-03.
- AI, M. (2024). Introducing llama 3.1: Our most capable models to date. *Meta AI Blog*.
- Bird, S., Klein, E., and Loper, E. (2009). Natural language processing with python: analyzing text with the natural language toolkit. " *O'Reilly Media, Inc.* ".
- Black, S., Hewitt, L. G., Dsouza, C. M., Biderman, S., Hallahan, M., McCourt, T., Uddin, Y., Vaidya, S., Gao, L., and Porraati, T. L. S. (2021). Gpt-neo: Large scale autoregressive language modeling with mesh-tensorflow. *EleutherAI*.
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.
- Contributors, U. (2023). Unsloth: A framework for efficient fine-tuning of large language models. <https://github.com/unsloth/unsloth>. Accessed: 2024-08-30.
- Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L. (2023). Qlora: Efficient finetuning of quantized llms. In *Advances in Neural Information Processing Systems*.
- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2018). Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.
- Doyle, A. C. (2003). *The Adventures of Sherlock Holmes*. Project Gutenberg. EBook No. 1661.
- Gerganov, G. (2023). llama.cpp: Port of facebook’s llama model in c++. <https://github.com/ggerganov/llama.cpp>. Accessed: 2024-09-03.
- Gholami, A., Kim, S., Zolnouri, Z. Y., and Mahoney, M. W. (2021). A survey of quantization methods for efficient neural network inference. *arXiv preprint arXiv:2103.13630*.
- Houlsby, N., Giurgiu, A., Jastrzebski, S., Morrone, B., De Laroussilhe, Q., Gesmundo, A., Attariyan, M., and Gelly, S. (2019). Parameter-efficient transfer learning for NLP. In Chaudhuri, K. and Salakhutdinov, R., editors, *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2790–2799. PMLR.
- Hu, E. and other authors (2021). Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.

- Huang, Q., Zhang, Y., Ko, T., Liu, X., Wu, B., Wang, W., and Tang, L. (2022). Personalized dialogue generation with persona-adaptive attention.
- Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., de las Casas, D., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., et al. (2023). Mistral 7b. *arXiv preprint arXiv:2310.06825*.
- Johnson, O. and Yi, L. K. (2024). Addressing copyright infringement and challenges in ai training. *Chambers and Partners*.
- Madotto, A., Lin, Z., Wu, C.-S., and Fung, P. (2021). Few-shot persona-grounded dialogue via contrastive fine-tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 9185–9196, Online. Association for Computational Linguistics.
- Mikolov, T., Chen, K., Corrado, G., and Dean, J. (2013). Efficient estimation of word representations in vector space. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- Pennington, J., Socher, R., and Manning, C. (2014). Glove: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67.
- Richardson, L. (2007). Beautiful soup.
- Riviere, M. et al. (2024). Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*.
- Sanh, V., Debut, L., Chaumond, J., and Wolf, T. (2019). Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. In *Advances in neural information processing systems*, pages 4467–4476. NeurIPS.
- Scaffidi, C. (2021). clean-text.
- Simpson, E. (2020). Generating persona-consistent dialogue responses using deep reinforcement learning.
- Touvron, H. et al. (2023). Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.

-
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. In *Advances in neural information processing systems*, pages 5998–6008. NeurIPS.
- von Werra, L., Belkada, Y., Tunstall, L., Beeching, E., Thrush, T., Lambert, N., and Huang, S. (2020). Trl: Transformer reinforcement learning. <https://github.com/huggingface/trl>.
- Wang, B., Biderman, S., Hallahan, M., Gao, L., Black, S., Golding, L., Hopkinson, E., Leahy, C., McDonell, K., and McElwee, S. (2021). Gpt-j: Large scale autoregressive language modeling with mesh-tensorflow.

Appendices

The repository can be accessed at <https://github.com/mustafameh/Sherlock-LLM>
However, since there is a file upload limit on GitHub and Moodle, the finetune model namely V5Sherlock8bit.gguf should be downloaded from https://drive.google.com/drive/folders/1SrBH66qNq7k65IUJy6nAFWot1w67B_lw?usp=sharing.
A short Demo Video is uploaded on Youtube <https://youtu.be/K7Yxn6PkKmc>